

电子科技大学

UNIVERSITY OF ELECTRONIC SCIENCE AND TECHNOLOGY OF CHINA

专业学位硕士学位论文

MASTER THESIS FOR PROFESSIONAL DEGREE



论文题目 基于 Zynq 平台的 Linux 实时性研究
及在 FC 网络中的应用

专业学位类别 工 程 硕 士
学 号 201822010211
作 者 姓 名 王 宁
指 导 教 师 谢 军 副教授

分类号 _____

密级 _____

UDC ^{注 1} _____

学 位 论 文

基于 Zynq 平台的 Linux 实时性研究及在 FC 网络中的应用

(题名和副题名)

王 宁

(作者姓名)

指导教师

谢 军

副教授

电子科技大学

成 都

(姓名、职称、单位名称)

申请学位级别 硕士 专业学位类别 工 程 硕 士

工程领域名称 电子与通信工程

提交论文日期 2021.05.17 论文答辩日期 2021.05.31

学位授予单位和日期 电子科技大学 2021 年 6 月

答辩委员会主席 _____

评阅人 _____

注 1：注明《国际十进分类法 UDC》的类号。

Linux real-time research on Zynq and its application in FC

A Master Thesis Submitted to
University of Electronic Science and Technology of China

Discipline: **Master of Engineering**

Author: **Wang Ning**

Supervisor: **Prof.Xie Jun**

School: **School of Information and Communication
Engineering**

摘要

在航电地面仿真系统中,普遍使用 FC 网络进行设备间高速的数据传输。系统中的 FC-AE-ASM 节点卡目前使用“x86+FPGA”分立式软硬件架构实现,该架构采用传统的“先硬件后软件”的设计模式,过程中需要反复迭代,开发周期长且成本高,经常在软硬件联调时出现不可预期的错误。

本文给出了一个基于 Xilinx Zynq 全可编程平台的 FC-AE-ASM 节点卡设计方案,Zynq 平台在单个芯片中集成了双核 ARM Cortex-A9 处理器和 FPGA,两者通过 AXI 片内总线连接,与“x86+FPGA”架构相比,明显提高了硬件利用率,降低了功耗,且使得软硬件之间的设计更具有协同性和灵活性。另外,由于航电地面仿真系统对操作系统的实时性有较高的要求,而 Linux 从诞生到目前的发展,一直被用作通用分时操作系统,实时性没有得到足够的重视。故本文的另一个重点是构建基于 Zynq-7000 SoC 的实时 Linux 系统,以满足设计需求。本文的主要工作如下:

1. 根据研究背景和工程需求,分析限制 Linux 系统实时性的主要因素,采用 Preempt-RT patch 的方式实时化改造 Linux 内核。
2. 完成数据交互通道的设计:对比几种不同的数据传输方式,选择使用 AXI_HP 接口和 PL 端 DMA 的方式实现软硬件数据的传输。
3. 在 Linux 设计架构下,设计节点卡驱动程序。配合硬件完成 FC 帧的接收与发送,并提供控制机制实现应用层对硬件的控制。
4. 设计节点卡应用程序,主要功能包括与总控机建立通信,解析并执行总控机的命令。
5. 对改造后的实时 Linux 和标准 Linux 进行任务响应延迟测试并对比,其中实时 Linux 的最大响应延迟仅为 69 μ s,验证了实时改造的有效性。同时搭建测试平台,对 FC-AE-ASM 节点卡进行性能和功能测试,验证了其在传输速率满足工程需求的同时,可以完成数据传输功能。

关键词: Linux 实时性, Xilinx Zynq-7000 SoC, FC-AE-ASM 节点卡

ABSTRACT

In avionics ground simulation systems, FC networks are commonly used for high-speed data transmission between devices. The FC-AE-ASM node card in the system is currently implemented using the "x86+FPGA" software and hardware design architecture. This architecture uses a discrete design model of "hardware first, software second", which requires repeated iterations, is time-consuming and costly, and often unpredictable errors occur during joint debugging.

This paper presented a FC-AE-ASM node card design scheme based on Xilinx Zynq fully programmable platform. The Zynq platform integrated a dual-core ARM Cortex-A9 processor and FPGA in a single chip, which were connected through the AXI on-chip bus. Compared with the "x86+FPGA" architecture, it significantly improved hardware utilization, reduced power consumption, and made the design between software and hardware more synergistic and flexible. In addition, For the higher requirements for the real-time operating system of avionics ground simulation system, while Linux has been generally used as time-sharing operating system, whose real-time performance has not been paid enough attention. Therefore, another focus of this article was to build a real-time Linux system based on Zynq-7000 SoC to meet the design requirements. The main work of this paper is as follows:

1. According to the research background and engineering requirements, analyze the main factors that limit the real-time performance of the Linux system, and use the Preempt-RT patch method to transform the Linux kernel in real time.
2. Design the data exchange channel: Compare several different data transmission methods, use the AXI_HP interface and PL side DMA to realize the transmission of software and hardware data.
3. Design the node card driver under the Linux framework. Cooperate with the hardware to complete the reception and transmission of FC frames, and provide a mechanism to control the hardware at the application layer.
4. Design the node card application program. The main functions include establishing communication with the master control machine, analyzing and executing the commands of the master control machine.
5. Test and compare the task response delay between the modified real-time Linux

and standard Linux. The result showing that the maximum response delay of real-time Linux is only 69 μ s, which verified the effectiveness of the real-time transformation. At the same time, built a test platform to test the performance and function of the FC-AE-ASM node card, the result of which verified it's data transmission function while the transmission rate meets the engineering requirements.

Keywords: Linux real-time, Xilinx Zynq-7000, FC-AE-ASM node card

目 录

第一章 绪论	1
1.1 研究背景与意义	1
1.2 航电地面仿真系统概述	2
1.3 Linux 实时化研究现状	4
1.4 研究内容与章节安排	5
第二章 实时操作系统与关键技术分析	7
2.1 实时系统概述	7
2.2 实时操作系统	8
2.2.1 实时操作系统概述	8
2.2.2 实时操作系统性能指标	9
2.3 Zynq 开发平台	10
2.3.1 Zynq-7000 总体结构	11
2.3.2 Zynq 平台开发环境介绍	13
2.3.3 Zynq 开发流程	14
2.4 AXI 总线介绍	16
2.4.1 AXI 通道介绍	17
2.4.2 AXI 握手机制	19
2.4.3 Zynq AXI 接口	19
2.5 FC 相关协议分析	21
2.5.1 FC 协议概述	21
2.5.2 FC-AE-ASM 协议分析	24
2.6 本章小结	26
第三章 Linux 实时性分析与实时化改造	27
3.1 Linux 实时性研究	27
3.1.1 Linux 系统概述	27
3.1.2 Linux 的实时性限制	28
3.2 Linux 的实时化改造方案	29
3.2.1 双内核方案	29
3.2.2 修改内核源码方案	30
3.3 Preempt-RT 的关键技术研究	31

3.3.1 优先级继承	31
3.3.2 临界区可抢占	32
3.3.3 中断线程化	33
3.4 Preempt-RT 的配置与移植	34
3.4.1 交叉编译环境的搭建	34
3.4.2 Preempt-RT 的配置与移植测试	35
3.5 本章小结	36
第四章 FC 节点卡软件的设计	38
4.1 航电地面仿真系统的构成	38
4.2 FC 节点卡整体架构	39
4.3 数据交互通道方案选择与设计	40
4.3.1 数据交互通道方案的选择	40
4.3.2 数据交互通道的设计	41
4.4 节点卡驱动程序的设计	49
4.4.1 初始化模块	50
4.4.2 数据发送模块	54
4.4.3 数据接收模块	56
4.4.4 控制模块	58
4.5 节点卡应用程序的设计	59
4.6 本章小结	60
第五章 测试与分析	62
5.1 Linux 实时性测试	62
5.1.1 Cyclicttest 测试	62
5.1.2 测试结果分析	63
5.2 节点卡软件测试	64
5.2.1 测试平台介绍	64
5.2.2 测试方案	65
5.2.3 收发功能测试	66
5.2.4 传输性能测试	67
5.3 本章小结	69
第六章 总结与展望	70
6.1 总结	70
6.2 展望	71

目 录

致 谢	72
参考文献	73
攻读硕士期间的科研项目和成果	76

图目录

图 1-1 舰载航电系统示意	2
图 1-2 航电地面仿真系统结构	3
图 1-3 分立式架构与 Zynq 平台架构	4
图 2-1 实时系统与嵌入式系统	7
图 2-2 任务切换时间示意	9
图 2-3 中断响应示意	10
图 2-4 Zynq 7000 芯片内部结构	11
图 2-5 软硬件协同设计流程	14
图 2-6 Zynq 片上系统设计流程	15
图 2-7 AXI4 读交易过程	18
图 2-8 AXI4 写交易过程	18
图 2-9 AXI 读写过程信号依赖图	19
图 2-10 PS-PL 互联结构	20
图 2-11 FC 网络支持的三种拓扑结构	22
图 2-12 FC 协议层次结构	23
图 2-13 FC 帧格式	23
图 2-14 FC 帧头格式	24
图 2-15 FC-AE-ASM 扩展帧头格式	25
图 3-1 arch 目录结构	27
图 3-2 GNU/Linux 系统架构	28
图 3-3 RTAI 与 Xenomai 方案架构对比	30
图 3-4 进程优先级以及到达时刻示意	31
图 3-5 进程优先级反转示意	32
图 3-6 优先级继承后执行过程	32
图 3-7 标准 Linux 内核中的中断处理过程	33
图 3-8 实时 Linux 内核中的中断处理过程	33
图 3-9 Linux 不同版本的补丁目录	35
图 3-10 Fully Preemptive Kernel(RT)选项配置界面	36
图 3-11 Linux 启动界面	36
图 4-1 系统总体设计架构	38

图 4-2 FC 节点卡架构	39
图 4-3 数据交互通道整体结构图	42
图 4-4 数据交互通道硬件工程图	43
图 4-5 Zynq-7000 PS IP 核	44
图 4-6 AXI DMA IP 核	46
图 4-7 AXI DMA 参数设置界面	47
图 4-8 AXI DMA IP 核地址分配	47
图 4-9 AXI BRAM Controller IP 核	48
图 4-10 Block Memory Generator IP 核	48
图 4-11 BRAM 自动分配的地址信息	48
图 4-12 Linux 系统中软硬件跟驱动的关系	49
图 4-13 驱动跟软硬件的联系	50
图 4-14 初始化流程图	50
图 4-15 帧数据发送程序流程图	56
图 4-16 帧数据接收程序流程图	57
图 4-17 控制模块交互结构	58
图 4-18 自定义 TCP 数据段格式	59
图 4-19 总控机 TCP 通信过程	60
图 4-20 节点卡 TCP 通信过程	61
图 5-1 Cyclicttest 测试结果柱状图	63
图 5-2 测试平台结构	64
图 5-3 Zynq 开发板 MZ7035	65
图 5-4 ASM 节点卡管理界面	65
图 5-5 ASM 帧生成配置	66
图 5-6 ASM 帧配置	67
图 5-7 ASM 帧收发统计	67
图 5-8 接收端帧解析	68

表目录

表 2-1 MZ7035 硬件资源	10
表 2-2 AXI 总线接口应用场景	16
表 2-3 Zynq AXI 协议读写通道信号	17
表 2-4 AXI 互联接口	21
表 2-5 FC-AE-ASM 帧结构	25
表 3-1 交叉编译工具链的组成要素	34
表 4-1 Zynq-7000 的数据传输方法比较	41
表 4-2 Zynq-7000 PS IP 核接口描述	45
表 4-3 AXI DMA IP 的主要接口说明	46
表 4-4 为 AXI DMA 的主要寄存器的具体说明	52
表 4-5 部分控制码列表	59
表 5-1 Cyclicttest 测试结果	63
表 5-2 FC-AE-ASM 传输速率测试统计	68

缩略词表

缩写	英文全称	中文全称
ADEOS	Adaptive Domain Environment for Operating System	操作系统自适应域环境
ANSI	American National Standards Institute	美国国家标准学会
ASIC	Application Specific Integrated Circuit	专用集成电路
ASM	Anonymous Subscriber Message	匿名用户信息
AXI	Advanced eXtensible Interface	高级可扩展接口
BRAM	Block Random Access Memory	块随机存取存储器
CRC	Cycle Redundancy Check	循环冗余校验
DDR	Double Data Rate	双倍数据速率
DMA	Direct Memory Access	直接寄存器访问
EMMC	Embedded Multi Media Card	嵌入式多媒体卡
EOF	End of Frame	帧结尾
FC	Fiber Channel	光纤通道
FC-AE	Fiber Channel Avionics Environment	光纤通道航空环境
FIFO	First Input First Output	先进先出
FPGA	Field Programmable Gate Array	现场可编程逻辑门阵列
FSBL	First Stage Boot Loader	第一阶段引导加载程序
fsmlabs	finite state machine labs	有限状态机实验室
GIC	General Interrupt Controller	通用中断控制器
HDMI	High Definition Multimedia Interface	高清多媒体接口
IDE	Integrated Development Environment	集成开发环境
JTAG	Joint Test Action Group	联合测试工作组
MM2S	Memory Mapping to Stream	内存映射到数据流转换
NC	Network Controller	网络控制器
OSADL	Open Source Automation Development Lab	开源自动化开发实验室
PL	Programmable Logic	可编程逻辑
PS	Processing System	处理系统
RISC	Reduced Instruction Set Computing	精简指令集

缩写	英文全称	中文全称
RTAI	Real Time Application Interface	实时应用接口
RTOS	Real Time Operation System	实时操作系统
S2MM	Stream to Memory Mapping	数据流到内存映射转换
SDK	Software Development Kit	软件开发环境
SoC	System on Chip	片上系统
SLCRs	System-Level Control Registers	系统级控制寄存器
SOF	Start of Frame	帧头

第一章 绪论

1.1 研究背景与意义

传统的嵌入式系统通常采用软硬件独立开发、联合调试的设计方法,这种设计模式由于没有一个统一的设计标准和开发环境,存在开发周期长、系统性能不稳定和开发成本高等缺点,经常在开发的过程中逐渐偏离最初的目标。软硬件协同设计的出现解决了这种传统设计模式的弊端。现代软硬件协同架构的基本定义是:一个处理器和一个或多个专用集成电路通过某种总线连接并且共享存储设备和寄存器的架构。专业集成电路完成各种并行计算的任务,同时在纯硬件电路上表现吃力的控制任务交给处理器完成^[1]。在已经问世的软硬件协同开发平台中,著名的可编程门阵列厂商 Xilinx 发布的 Zynq-7000 系列核心产品是最优秀的代表之一,Zynq-7000 soc 是一个将多核 Cortex 处理器和 FPGA 集成在一个芯片的嵌入式开发平台,其中 Cortex 处理器可以在嵌入式操作系统的支持下,驱动各个硬件接口并且运行复杂度较高的控制命令,FPGA 则可以高效完成海量的并行计算,二者各司其职且在需要沟通时通过一定的机制相互通信,配合合理的设计思路可以将异构多核平台的优势最大化。对于 Zynq-7000 平台的详细介绍将在第二章进行。

由于软硬件协同技术和这种异构多核平台逐渐应用到了实际生产中,平台处理器上任务的最差响应延迟逐渐得到业内的关注。毫无疑问,在这一点上具有强实时特性的操作系统表现是更加抢眼的,业内公认实时操作系统运算结果的正确性取决于两个关键因素:一是正确的响应,二是该响应在严格的受限时间内得到。这个特性在一些关键性的领域中是必需的,如航电系统、汽车安全系统等。

ARM 系列处理器作为世界上最先进的 RISC(Reduced Instruction Set Computing,精简指令集)微处理器之一,具有性能优越、功耗小、成本低、接口丰富等优点^[2]。Xilinx Zynq-7000 中的 ARM Cortex-A9 处理器是基于 ARM v7 架构且能与其他 Cortex 系列处理器兼容的一款高性能处理器^[3]。为处理器选择操作系统时,要考虑的因素包括使用需求、开发成本和硬件支持程度等。在大多数关键的应用领域中,常见的操作系统包括 VxWorks、 μ C/OS-II、 μ Clinux、DJYOS 和 Linux 等,其中 VxWorks 系统因其出众的实时特性成为目前嵌入式系统领域中应用最广泛的实时操作系统之一,如 FC-AE 交换机,采用的是 PowerPC+FPGA 架构进行软硬件协同开发,PowerPC 处理器上移植的就是 VxWorks 操作系统^[4]。对于 ARM 处理器来说,使用较多的系统有 μ Clinux、 μ C/OS-II 和嵌入式 Linux, μ Clinux 是一款内存占用很小的操作系统,仅支持没有内存管理单元的小型处理器,而 μ C/OS-II 本身支持的内核功能也比较局限,而且不是免费使用的。嵌入式 Linux 有两种常见的类

1.2 航电地面仿真系统概述

舰载航电系统的模拟见图 1-1，系统中存在着很多需要进行数据交互的航空电子设备（称为节点），这些节点和交换机连接，通过机载数据总线来传送数据。

图 1-1 舰载航电系统示意

由于在实际的航电环境中，军事指挥系统和监控系统使用的都是 FC 网络和相关协议，如果性能和正确性出现漏洞后果会非常严重。为了研究航电系统的性能指标和保障数据交互的效果，在实验室中搭建其对应的地面仿真系统进行提前演示和改进是非常必要的。目前的地面仿真系统结构见图 1-2，系统中的几个核心设备均可以使用软硬件协同设计方法代替传统的分立式系统设计，如 FC-AE 交换机和各种仿真节点卡。图 1-2 种的节点卡从功能角度上可划分为监控卡、故障注入卡、雷达仿真卡等，从支持的协议角度上可分为 FC-AE-ASM 节点卡和 FC-AE-1553 节点卡等。

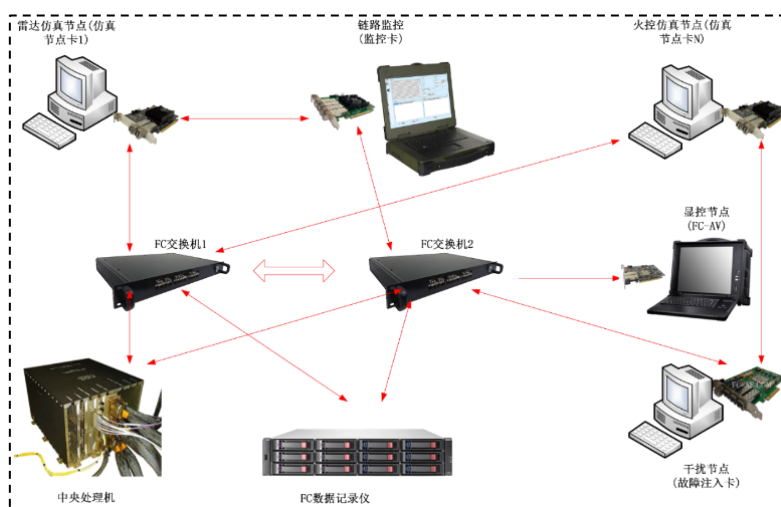


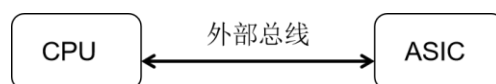
图 1-2 航电地面仿真系统结构

如图 1-2 所示，整个系统由 FC 网络交换机、FC 仿真节点卡和 FC 数据记录仪等设备组成，对于 FC 网络交换机来说，目前我们教研室使用的是 PowerPC +FPGA 的分立式架构的设备实现的，PowerPC 处理器通过 PCIe 总线和 FPGA 连接，进行数据交互和寄存器配置，FPGA 完成各个端口的数据转发工作，PowerPC 在 Vxworks 操作系统的协助下完成对 FPGA 的控制和请求帧的解析和应答等工作。同样地，FC 仿真节点卡是基于 x86+FPGA 分立式架构设计的，首先根据 FC-AE 相关协议在 FPGA 上实现节点卡的基本功能，然后通过 PCIe 接口将 FPGA 板插到装有 Linux 或者 window 系统的 PC 上，在 PC 上开发与总控机通信的节点卡软件（驱动程序和应用程序）。值得注意的是，移植了实时 Linux 系统的 Zynq 平台在以上两种设备中都有很突出的应用价值。

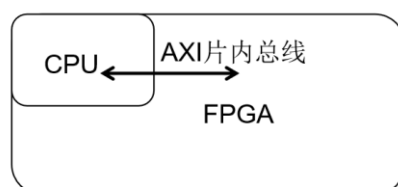
本文的研究重点之一是将目前分立式架构设计的 FC 节点卡使用 Zynq 平台代替。如图 1-3 所示，分立式架构设备的软件设计部分运行在 CPU 上，通过外部总线与 ASIC 进行通信。Zynq 平台的结构则不同，CPU 和 ASIC 不再是两个独立的

元件，而是被集成在一个芯片内，通过 AXI 片内总线连接，相比于使用分立式架构的优势如下：

1. 具有更小的设备体积和更大的硬件资源利用率。
2. 可以使用 Xilinx 软硬件协同开发工具套件进行开发，使得整个设计过程更具有协同性和灵活性。
3. 片内总线连接，使得 CPU 和 FPGA 之间的通信更具稳定性和安全性。



传统分立式结构



Zynq平台结构

图 1-3 分立式架构与 Zynq 平台架构

1.3 Linux 实时化研究现状

实时操作系统出现的很长一段时间内都没有得到足够的关注，仅仅用在一些实时性要求苛刻的关键系统中，在其他领域还未崭露头角^[6]。一直到二十世纪初，绝大多数领域中的嵌入式处理器上才都移植了实时操作系统，这也使得实时操作系统有巨大的市场需求，吸引了世界上顶尖的科技公司 and 顶尖高校的研究人员进入研究和开发嵌入式实时操作系统的项目中来，并相继推出了很多成熟的产品^[7]。目前，世界上先进的代表产品包括美国风河公司的 VxWorks、Lineo 公司的 uClinux、Microsoft 公司的 Windows CE 和美国新墨西哥州的 fsm labs (finite state machine labs, 有限状态机实验室) 的 RTLinux 等。其中 VxWorks 系统是一款硬实时操作系统，支持多种处理器架构，主要应用在实时性要求较为苛刻的领域，如美国的“好奇号”探测器。值得注意的是，由于硬件设备的发展，目前硬件性能的提升让系统延迟降低到用户可接受程度，原来很多使用昂贵的商业硬实时操作系统的设备已经改用 Linux 或者实时化改造后的 Linux，所以 Linux 在很多关键性领域的系统上得到迅速推广，同时也使得它的实时性在业内得到广泛关注，变成了其发展的一大趋势。

目前,很多主流的公司和组织从 Linux 源码设计的角度分析其实时性能^[8],已经提出了很多解决方案,这其中较为出色的包括: Kurt-Linux、RED-Linux、Xenomai/RTAI 和 Preempt-RT。

Kurt-Linux 是由 University of Kansas 在 Linux 架构的基础上开发的第一个实时操作系统,但是由于它有一些较为显著缺点,即实时进程经常会受到非实时进程的影响,在很多特定的场景下会丧失硬实时特性。所以目前主要应用在音视频处理器中。

RED-Linux 系统和 Linux 内核相比主要的区别是实现了一个通用实时调度框架。该框架可以支持对驱动实时化的一些调度策略,这也是 RED 的一大亮点^[9]。

Xenomai 和 RTAI 是两个实时微内核,为了解决积怨已久的 RTLinux 移植困难的问题提出。2003 年,这两个独立的项目进行合并研发,不过在两年以后又再度分开,有了不同的发展轨迹。RTAI 追求极致的实时性,但是对很多嵌入式设备的支持还远远不够,其技术社区也不活跃。Xenomai 则相反,它对于大多数系统都具有兼容性,技术社区活跃。目前这两个实时微内核都是在第三方软件平台上实现的,对其详细的介绍将在第三章进行。

Preempt-RT 是一个出色的修改 Linux 内核源码的项目,从 2004 年项目启动开始到现在,以 Ingo Molnar 为主的项目组成员一直在维护和更新各种版本。Preempt-RT 在将 Linux 内核转换为实时操作系统上的优秀表现,得到内核开发人员的认可,并且在多次成为 Linux 内核峰会上的重点讨论对象,而且还为了更好的接受 Preempt-RT 对内核做出一些修改。Preempt-RT 项目一直都很活跃,在 2009 年到 2013 年期间得到过 RedHat 公司资金支持,2014 年完全由业余爱好者兼职维护和开发,2015 年 OSADL 继续提供资金保障 Preempt-RT 的维护。从 2016 年以来,Thomase 的 Preempt-RT 研发组开发的几百个 patch 被收入主线 Linux 中,这对于 Preempt-RT 来说意义重大,不仅大大减小了 Preempt-RT 的代码量,而且改善了 Linux 内核的部分结构,使得 Preempt-RT 的移植更加方便。

1.4 研究内容与章节安排

本文的主要研究内容包括:针对 Linux 系统实时性的欠缺,对 Linux 做实时化改造并在 Zynq-7000 SoC 上搭建实时 Linux 系统;同时设计 FC-AE-ASM 节点卡软件,包括数据交互通道部分的设计、FC 节点卡驱动程序设计和 FC 节点卡应用程序设计;最后进行测试与分析,主要包括 Linux 实时性测试和节点卡测试两个部分。

各章节安排如下:

第一章 绪论。首先提出论文研究背景与意义，同时介绍了本文涉及的航电地面仿真系统和 Linux 实时化研究现状，最后对各章节安排和工作内容进行阐述。

第二章 实时操作系统与关键技术分析。本章主要对论文涉及到的一些背景技术和相关协议进行归纳和介绍。首先给出了实时系统和实时操作系统的基本概念，然后介绍了 Zynq 平台和 AXI 总线。最后，提及了本文涉及到的 FC 相关协议，主要内容包括协议的特点和帧头结构等。

第三章 Linux 实时性分析与实时化改造。本章的主要内容包括 Linux 实时性研究、Preempt-RT 关键技术分析和 Linux 实时化改造等。首先，由于目前 Linux 的标准版本很难达到航电地面仿真系统对操作系统实时性的要求，在分析了内核实时性限制因素后，针对这些因素提出使用 Preempt-RTpatch 的方法改进。最后将改进了的实时 Linux 移植到 Zynq 目标平台上。

第四章 FC 节点卡软件的设计。首先介绍航电地面仿真系统的整体结构，明确 FC 节点卡在该系统中的功能。在此基础上给出基于 Zynq 的 FC 节点卡设计架构，明确设计目标并进行任务划分。最后进行软件部分的设计，主要设计内容包括数据交互通道部分设计、FC 节点卡驱动设计和 FC 节点卡应用程序设计。

第五章 测试与分析。测试主要分为两个方面：一是 Linux 实时性测试，首先需要了解 Cyclicttest 实时测试工具的工作原理，然后通过测试最大任务响应延迟时间这一重要指标，对改造后的实时 Linux 和标准 Linux 在同一硬件平台上对比，证明 Preempt-RT patch 改造的有效性。二是 FC 节点卡测试，包括帧收发功能测试和传输性能测试，收发功能测试即测试其能否正确完成 FC-AE-ASM 帧的发送与接收。传输性能测试即测试在不同的链路速率下，发送速率是否可以达到预期目标。

第六章 总结与展望。本章系统地总结了论文研究内容，同时针对目前的一些不足之处提出了今后继续研究的方向。

第二章 实时操作系统与关键技术分析

本章将详细介绍整文涉及到的一些背景知识和关键技术，包括实时系统概述、实时操作系统、Xilinx Zynq 平台和 AXI 总线，最后介绍 FC 协议和 FC-AE-ASM 协议。

2.1 实时系统概述

实时系统指的是对于任意输入都能在一个有限时间给出响应的系统^[10]。在实时系统中，超出系统规定的时限产生的结果都认为是错误的，即使输出的响应正确^[11]。这里的实时并不要求对外界输入反应灵敏，它追求的是稳定性和可预测性，即对于外部输入在规定的最晚时限内得到相应即可。

根据系统的实时性能不同，在具体的应用中经常会被区别为硬实时和软实时系统^[12]。但是两种区分的界限一直比较模糊，业内目前还没有明确的指标和测试方法可以断定某个具体的系统究竟属于哪个类型。一般而言，如果系统在绝大多数情况下都能保证其上的关键任务都能在一个较小的时限内执行完毕，即是硬实时系统，比如飞机或汽车的自动驾驶系统。而对于软实时系统，比如网络通话系统、服务型机器人和数控系统，这种系统不会过度依赖于实时性且一般有一个容错区间，在区间内即使发生了延迟错误也会正常的运行^[13]。

如果一个实时系统中的开发平台具有很强的专用性，如 Zynq 平台，那么这个系统同样也存在嵌入式特性。“实时”和“嵌入式”是从不同的角度出发对系统的定义，其关系见图 2-1。

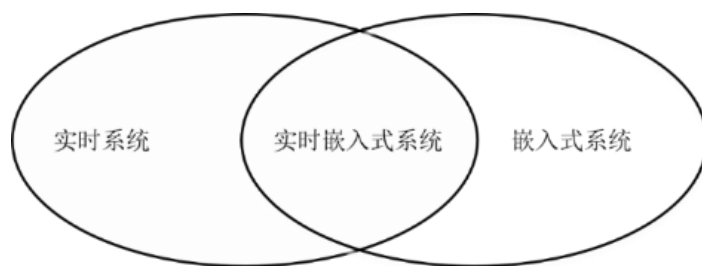


图 2-1 实时系统与嵌入式系统

2.2 实时操作系统

2.2.1 实时操作系统概述

通常来说, 实时操作系统 (Real Time Operation System, RTOS) 可以在重负载的情况下及时响应关键任务的请求。当有关键任务就绪时, RTOS 会抢占非关键任务的资源 (如 CPU) 来保证关键任务的响应。RTOS 并不是追求高的吞吐率, 相反往往会牺牲一定的吞吐率来保证系统的可预测性, 即在任何条件下都能保证关键任务在规定最晚时限得到执行^[14]。一个 RTOS 的前提是需要达到通用操作系统的功能和特性, 除此之外还应满足以下基本特征:

1. 具有足够的优先级。在 RTOS 中足够多的优先级划分是进行调度的前提, 可以用来区分不同级别的任务, 将关键任务凸显出来。
2. 具有多任务调度和抢占机制。RTOS 的调度器应该允许系统中多个任务之间的抢占执行, 这样才能保证优先级更高的关键任务比非关键任务更早的执行。
3. 具有实时调度策略。当实时任务越来越多时, 如何高效的给实时任务分配 CPU 时间由实时调度策略来决定, 一个好的实时调度策略首先应该具有较低的时间复杂度。
4. 截止时限裁决功能。实时操作系统应能够动态的识别关键任务的截止时限, 同时为了更加有效的处理, 还可以将时限转化为所有的资源都可分配使用的优先级信息。

在实际应用中普遍区分分时和实时两种操作系统, 这两种操作系统的界限是很明显的, 因为它们的设计理念完全不同, 前者追求资源公平划分和公平调度, 后者则追求系统在执行任务特别是关键任务时的确定性。同实时系统相同, 行业内也普遍习惯把 RTOS 区别为硬实时和软实时两种类型, 这两种类型划分同样没有明确界限, 但是可以以此区分实时操作系统是否足够优秀。对于在绝大多数恶劣条件下也可以保证关键任务在可接受的时限内执行的系统一般可归为硬实时操作系统, 同时如果超出时限会导致系统失败^[15]。软实时操作系统在小概率情况下不能保证关键任务能及时执行, 对于一些允许在少数情况下出现延迟较大的系统可以使用软实时操作系统^[16]。值得注意的是, 整个系统对收到的任务是否能快速回应是由硬件平台的性能和其上运行的软件共同决定的, 而且硬件平台毫无疑问是影响最大的。再出色的实时操作系统也无法在一个过时且资源欠缺的处理器上发挥它的优点, 所以在保证了硬件平台性能的前提下, 使用实时操作系统才是有意义的。同时如果需要评估和对比操作系统的实时性, 也需要保证是在同一个硬件平台上进行。

2.2.2 实时操作系统性能指标

为评估操作系统是否为实时操作系统（RTOS）或评估 RTOS 的实时性能，需要明确 RTOS 的实时性能指标，Rhealstone 是一个被普遍认可度的实时性能测量标准^[17]，它测量六个时间量作为关键指标，分别为上下文切换时间（Task Switching Time）、中断响应延迟时间（Interrupt Latency Time）、抢占时间（Preemption Time）、信号量混洗时间（Semaphore Shuffling Time）、死锁解除时间（Deadlock BreakingTime）、数据包吞吐率（Datagram Throughput Time）^[18]。其中，最主要的两个因素是上下文切换执行时间和任务响应延迟，业内普遍使用这两个标准判断操作系统的实时性。

1. 上下文切换时间

上下文切换时间定义为操作系统将 CPU 使用权在两个不同任务之间交接花费的时间^[19]。如图 2-2 所示，从新的调度任务发生起，整个时段包括保存 A 的上下文、选择下一个任务 B 执行和恢复 B 的上下文时间^[20]。上下文切换时间很大程度上受硬件因素的影响，除此之外，还取决于操作系统使用的数据结构和调度策略的时间复杂度^[21]。

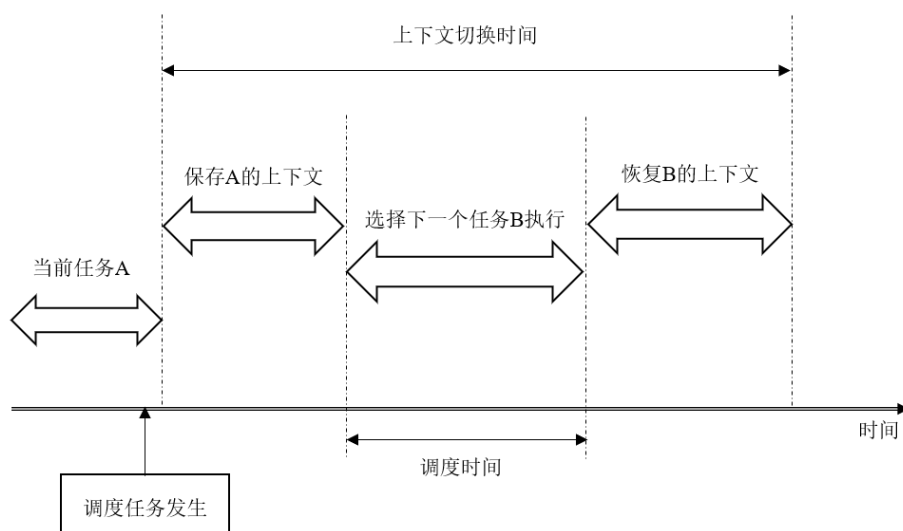


图 2-2 任务切换时间示意

2. 中断响应时间

中断响应时间指的是从中断产生到执行中断服务程序的第一行代码之间经历的时间。主要包括了中断响应延迟时间和保存处理器状态时间。整个中断响应过程见图 2-3，首先从中断发出到接收到请求有一个中断响应延迟时段，然后完成了三个动作：保存处理器状态、中断处理和中断恢复。其中中断处理时间和具体的中断

处理程序有关，进程调度时间和调度策略相关。中断响应延迟时间和操作系统性能相关性最大，且主要软件因素是最大关中断时间。

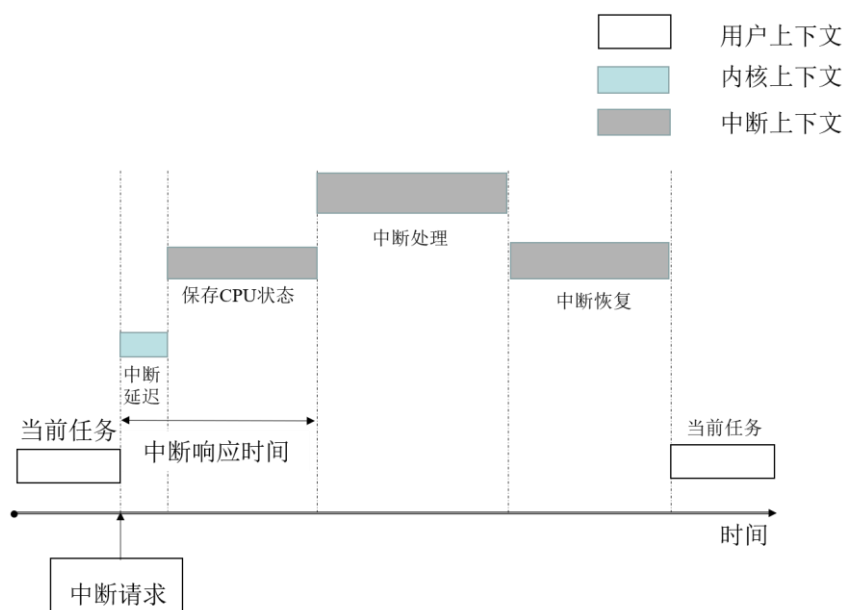


图 2-3 中断响应示意

2.3 Zynq 开发平台

Zynq-7000 作为一款全可编程片上系统被定义为：将运算和控制能力都很强的处理器和可编程逻辑集成在一个芯片中的全新的设计平台。在全球 IT 行业内，该平台都具有一定的先进性和独特性^[22]。Zynq 平台的诞生使得嵌入式系统的开发周期更短，系统整体性能明显提高，同时也把可编程逻辑的应用范畴扩展到了嵌入式领域。目前，Zynq-7000 SoC 系列下有多个已经上市的核心板型号，本文的设计使用的是米联客公司 MZ7035 开发板，该开发板中的 Zynq 芯片型号为 XC7Z035，表 2-1 中详细列出了 MZ7035 中的各种硬件资源和配置信息。

表 2-1 MZ7035 硬件资源

资源名称	具体参数
FPGA	XC7Z035FFG676-2I
PS 端 DDR3	1GB (2 片 512MByte)，数据带宽可达 1066MHZ*32bit
PL 端 DDR3	1GB (2 片 512MByte)，数据带宽可达 1600MHZ*32bit
FLASH	256Mbit QSPI FLASH
EMMC	8GB

资源名称	具体参数
时钟管理单元	核心板（XC7Z035）：一路 PL 端 100M 时钟；一路 PS 端 33.333M 时钟。 底板：一路可编程时钟（10-810MHZ 可编程）接口
电源管理单元	1.0V 核心电源，最大输出 30A
SFP+接口	4 路 SFP+接口，单路最大支持 10.3125Gbps
FEP 接口	96GPIO/48 对差分
千兆以太网	PS 端 1 路千兆网口
PCIE 接口	4 路 PCIE2.0 接口
HDMI	1 路 HDMI 输出，支持 1080P
USB HOST	4 路 USB HOST，可接 USB 外设
USB 串口	核心板 1 路 CP2104 USB 转串口，底板 1 路 USB 接口， 共用一颗串口芯片
TF 卡座	插入 SD 卡，SD 卡存储启动文件
JTAG 接口	使用下载器进行调试和下载
按键	核心板 1 个，底板 4 个
外型	核心板：70mm*62mm；底板：200mm*110mm
电源	DC-12V/2A

2.3.1 Zynq-7000 总体结构

Zynq 7000 芯片内部的详细结构如图 2-4 所示。

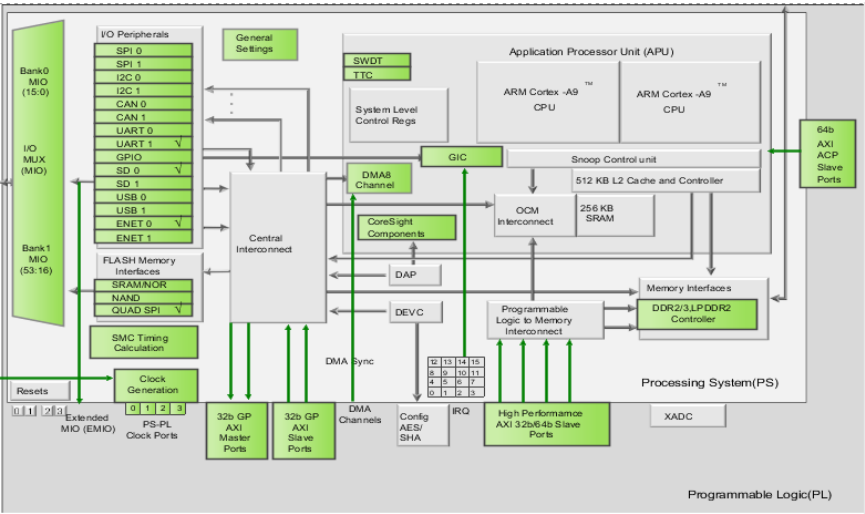


图 2-4 Zynq 7000 芯片内部结构

Zynq-7000 SoC 内部嵌入了 Cortex 处理器 (Processing System, PS) 和可编程逻辑 (Programmable Logic, PL)。其中 PS 是以 ARM 公司的 Dual-Core ARM Cortex-A9 处理器为核心, 同时包含了丰富的硬件资源; PL 即 FPGA, 这两个部分由片内的 AXI 总线进行连接。在 Zynq 平台中, PS 端可以作为一个独立的系统运行, 此时可以完全不使用 FPGA 资源。同时 PL 端也可以独立运行, 前提是启动时需要 PS 端的支持, 因为系统上电后首先运行的是 PS 部分的 Flash 存储器或者 SD 卡中的程序。当然也可以协同运行, 此时一般将 PL 端的硬件逻辑设计作为 PS 端的从设备或者外设, 比如 CPU 读写与 PL 连接的各种接口。下面分别对 PS 和 PL 具体介绍。

2.3.1.1 处理系统结构

PS 内部结构如图 2-4 白色背景部分, 其核心模块是应用处理单元 APU, 同时包括两个重要的模块分别为存储器接口和 I/O 外设。

1. 应用处理单元 APU

APU 嵌入了两个基于 ARM v7 架构的 Cortex-A9 处理器, 并提供了标准的 ARM 指令和 Thumb-2 指令, 以及 Jazelle RTC 和 Jazelle DBX JAVA 加速。两个 Cortex-A9 核都有独立的 NEON、32KB L1 指令高速缓存和 32KB L1 数据高速缓存, 同时共享带有校验的 512KB L2 高速缓存。一些其他重要的硬件资源如下:

- (1) 每个 Cortex-A9 有私有定时器和看门狗定时器。
- (2) 系统级控制寄存器 (System-Level Control Registers, SLCRs), 一组不同的控制器用来控制 PS 的行为。
- (3) 提供带有校验功能的 256KB 片上存储器 OCM。
- (4) 64 位高级可扩展接口 (Advanced Extended Interface, AXI), 具有访问 L2 高速缓存和 OCM 的能力, 以及保证在数据交易时与 L1 和 L2 高速缓存的数据一致性。
- (5) 8 通道 DMA 控制器, 其中 4 个用于 PS 内部系统, 实现其与系统内的存储器的数据传输; 另外 4 个用于 PS 与 PL 之间信息传输。
- (6) 通用中断控制器 (GIC), 具有独立的中断屏蔽和优先级。包括 5 个 CPU 私有中断、16 个 CPU 软件中断, 以及分别来自系统、PS 和 PL 剩余部分的共享外设中断 (其中 20 个来自 PL)。

2. 存储器接口

具有 DDR 控制器接口, 支持 DDR3、DDR2、LPDDR-2 类型的存储器, MZ7035FD 上集成了 2GB 大小的 DDR3, 其中 1GB 连接到 PS 端, 1GB 连接到 PL

端。同时还具有四-SPI 控制器和静态存储器控制器接口，MZ7035FD 上集成了 256Mbit QSPI FLASH 和 8GB 的 EMMC，MZ7035FD 的具体硬件配置见表 2-1。

2.3.1.2 可编程逻辑结构

可编程逻辑 (PL) 提供了具有可自定义配置能力的丰富硬件结构，下面分别列出并对本文涉及到的资源做简要介绍：

1. 可配置逻辑块

包含 6 输入具有存储能力的查找表。

2. 36Kb 容量的 BRAM 资源

BRAM 的配置和使用是本文的重点之一，它提供了双端口访问能力，两个端口可以分别连接 PS 部分和 PL 内逻辑模块，实现 PS 对 PL 内寄存器的控制。

3. 数字信号处理 DSP48E1 资源

4. 时钟管理单元

5. 可配置 I/O 资源

6. 低功耗的吉比特收发器

7. 模拟-数字转换器 (XADC)

8. 用于 PCI-E 设计的集成接口模块

2.3.2 Zynq 平台开发环境介绍

本设计中使用的 Zynq 平台开发环境是赛灵思的 Vivado 开发套件，该套件自发布以来更新了很多版本，本文在工程设计中使用的是 2017.4 版本，需要注意的是，新版本可以完全兼容旧版本，反之则可能会出现不可预期的问题。开发套件中主要包括两个软件，一个是用于可编程逻辑设计的 Vivado 集成开发环境 (Vivado IDE)，Vivado IDE 用于开发 PL 部分的逻辑功能和 PS 与 PL 之间的数据传输通道；另一个是用于 PS 端设计的软件开发环境 (Software Development Kit, SDK)，在本文中用于生成运行实时 Linux 操作系统需要的文件^[23]。

在 Vivado IDE 环境下做 FPGA 相关开发时，可以使用硬件描述语言，也可以使用块设计板块中的 IP 核 (也叫做模块)，对 IP 核进行配置、集成和验证。Vivado IDE 设计的硬件工程经过编译后会生成 .bit 类型的比特流文件，它描述了整个可编程逻辑的所有信息。将实时 Linux 配置并完成交叉编译后，为了移植到开发板需要将比特流文件、一级启动文件以及 uboot.elf 文件一起合成 BOOT.bin，将 BOOT.bin 下载到 QSPI Flash 或 SD 卡中。本文在 Vivado IDE 下完成数据交互通道部分的设计，过程中使用到了一些重要的 IP 核，如 Zynq-7000 PS 模块、AXI DMA 模块、

BRAM 模块、FIFO 模块等。

2.3.3 Zynq 开发流程

由于 Zynq 的开发过程是一个典型的软硬件协同设计过程，本小节首先介绍一下普适性很强的软硬件协同开发的方法流程，然后具体介绍 Zynq 平台的开发流程。

软硬件协同开发指的是软件和硬件配合实现预期功能和开发目标，如图 2-5，整个开发过程包括软硬件任务划分、软件编程、硬件设计和联合调试等。

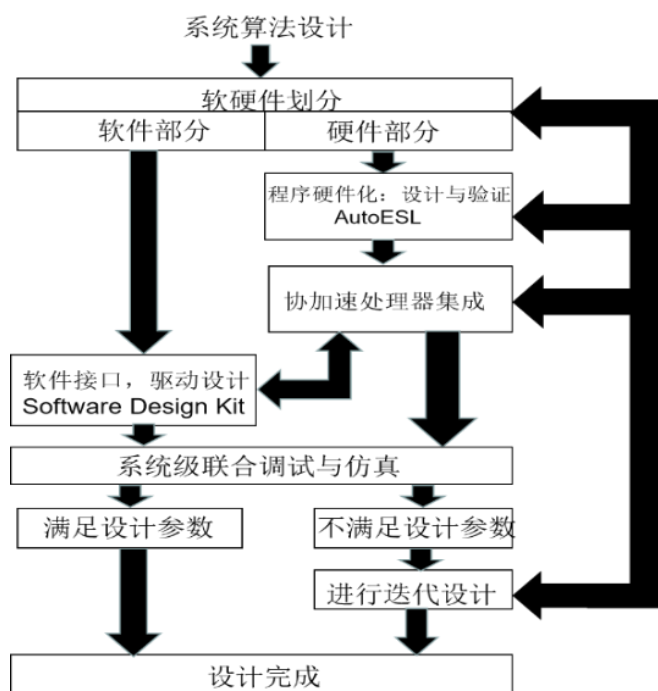


图 2-5 软硬件协同设计流程

整个流程的第一步是根据具体的设计要求描述出系统总体功能，提出设计指标，也可以细分出多个功能模块。接着进行软硬件任务分工，即一部分功能用高级编程语言描述并在处理器上执行，另一部分做硬件逻辑设计，如 FPGA 设计。这是整个系统设计过程中较为复杂的一步，且如何做好划分也没有相应的定论。从理论上讲，软件部分适合执行串行性或读写控制之类的任务，硬件逻辑电路适合完成复杂度较高的并行计算任务。在硬件部分和软件部分都设计并单独验证完成后将软硬件进行系统级别的联合调试，如果不满足开发预期则需要重新设计。

一般而言，软硬件联合调试是非常困难且不可避免的一步，常常导致整个设计过程不断返工重新设计，浪费大量人力物力。但是 Zynq 平台和配套开发套件的出现使得这一问题得到根本上的解决。

Zynq 片上系统详细的开发过程见图 2-6：

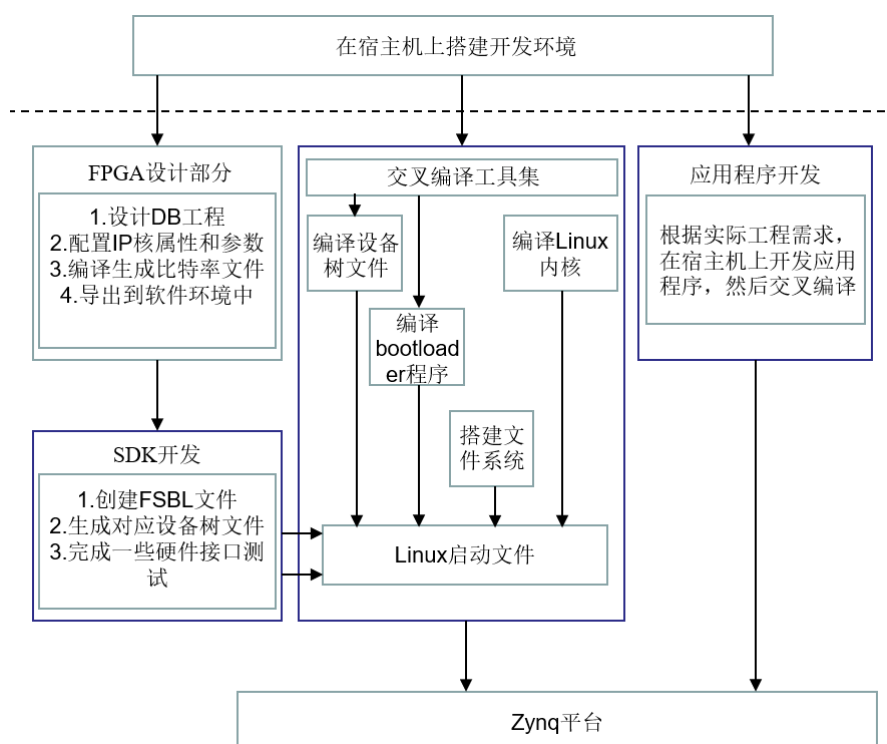


图 2-6 Zynq 片上系统设计流程

如前所述，Xilinx 提供的设计工具解决了传统软硬件联合开发的反复迭代问题。对应于前面的主要步骤，首先将软硬件部分进行具体任务分工，两种不同的任务要有较为明显的界限且需要明确 PS 和 PL 在工作过程中需要通信的内容和方式^[24]。对于硬件设计，Xilinx 提供了两种支持 ZYNQ 的逻辑设计模式，对于系统软件的开发则使用 Vivado 工具集中的 SDK 工具，可以对 PS 部分进行裸机开发，即创建裸机软件和裸机的硬件驱动程序，然后进行编译和调试。同时 SDK 中还包含了 GNU 交叉编译工具链，可以为 PS 部分中 ARM 处理器上移植操作系统提供帮助，比如生成系统启动所需要的 FSBL 文件。在移植操作系统的情况下，整个 Zynq 平台的启动过程如下：

1. 在 Vivado IDE 中对 PL 部分进行设计、配置、综合后得到比特率文件。
2. 搭建交叉编译环境，使用交叉编译器编译 U-BOOT、内核和设备树文件，分别得到 uboot.elf、uImage、devicetree.bit 三个必需的文件。
3. 在 SDK 中生成 FSBL 文件，然后把 uboot.elf 文件、FSBL 文件和比特流文件合成为 BOOT.bin 文件。
4. ZYNQ 设置为 SD 卡启动，devicetree.bit、uImage 和 BOOT.bin 放到 SD 卡中，进行启动测试。

2.4 AXI 总线介绍

高级微控制器总线结构是一款片上总线标准。目前为止，该总线的应用领域已经远超出微控制器范畴，ASIC 和 SoC 成为了它的主流应用方向，同时也用于便携设备的应用处理器和作为 SoC 内 IP 互联的规范中。高级微控制器总线结构在第三个版本中新增了高级可扩展接口（Advanced Extensible Interface, AXI）规范，主要适用于更高性能的片上互联。七年后，Xilinx 公司参与推出了 AXI4 规范并在 FPGA 中兼容了 AXI4 协议。

在 DMA 技术出现之前，CPU 只能使用轮询和中断控制的方式访问外设。这两种方式都有其明显的缺点，其中轮询方式需要 CPU 不断循环查询外设，如果查询到外设已经准备好读写数据，CPU 就可以对外设进行读写。这种访问方式会使得 CPU 在很长一段时间内都在等待外设数据的到达，导致其利用率低下。中断方式克服了这个缺点，它是通过硬件中断去通知 CPU 进行读写，这种方式在外设不忙的情况下既不影响 CPU 正常工作，也可以对外设数据及时处理，但是当大量数据传输时，CPU 会频繁中断，大量时间消耗在中断切换和保存进行状态上。直接存储器访问（Direct Memory Access，下文称 DMA）技术可以脱离处理器控制实现外设与存储器之间大量数据的直接传输，很好的解决了之前两种方式的问题。DMA 引擎运行少量的指令集，因此为指定的 DMA 传输提供了一种灵活的方法，用于对 DMA 引擎编程的软件程序写入到系统存储器中。通过使用 AXI 主接口，DMA 控制器就可以访问该区域。可以做 DMA 控制器中存储器映射的设备包括 DDR、线性寻址的四-SPI 读存储器、SMC 存储器和 PL 外设，或者连接到一个 M_GP_AXI 接口的外部存储器。

Zynq-7000 SoC 支持三种 AXI4 总线类型，三种总线均对应三种不同类型接口，分别是 AXI4、AXI-Lite 和 AXI4-Stream^[25]。它们适用于不同的场景，AXI4 接口在三种 AXI 接口中的传输速率最高，同时面向地址映射且支持最大 256 轮的猝发数据通信。AXI4-Lite 总线接口作为 AXI4 接口的一种精简版使用，一般出现在吞吐率要求低的地址映射传输。AXI4-Stream 主要应用在高速流数据传输，由于不提供地址相关的信号，不能进行地址映射。三种接口类型具体的应用场景见表 2-2。

表 2-2 AXI 总线接口应用场景

接口类型	应用场景
AXI4	嵌入式系统中的 AXI 监视器
	以太网接收器
	音视频 DMA 控制器

接口类型	应用场景
AXI4-Stream	某些计算机系统中的 Stream-FIFO
	通信网络领域的编解码器中
	音视频处理领域的流式音视频传输
AXI4-Lite	某些嵌入式设备中的存储器接口
	音视频处理中的 Gamma 校正
	嵌入式外部设备控制器

2.4.1 AXI 通道介绍

AXI4 包含了 5 个不同的通道，且在传输通道的两端有主设备和从设备之分，每个通道由一个信号集构成，Zynq 平台中 AXI 协议读写通道具体信号见表 2-3。

表 2-3 Zynq AXI 协议读写通道信号

	信号所属通道	信号名称	信号描述
读通道	读数据通道 (下文称 R 通道)	RDATA	表征要读的数据
		RVALID	表征读数据有效信号
		RREADY	表征读数据准备完毕信号
		RRESP	表征读事务处理状态信号
	读地址通道 (下文称 AR 通道)	ARADDR	表征要读的地址
		ARVALID	表征读地址有效信号
		ARREADY	表征读地址已准备完毕信号
		ARPROT	表征读地址通道安全标识
写通道	写响应通道 (下文称 B 通道)	BRESP	表征写事务处理状态信号
		BVALID	写响应有效，表征写命令的有效写入响应
		BREADY	响应准备，表示主机准备好接收一个响应信号
	写数据通道 (下文称 W 通道)	WDATA	表征要写的数据
		WREADY	表征写数据已准备完毕信号
		BRESP	表征写事务处理状态信号
		WVALID	写数据有效，表征可写入有效数据
		BVALID	写响应有效，表征写命令的有效写入响应
		BREADY	响应准备，表示主机准备好接收一个响应
		WSTRB	写选通，表示通道有有效数据

信号所属通道	信号名称	信号描述
写地址通道 (下文称 AW 通道)	AWADDR	写地址
	AWVALID	表征该通道可写入有效的地址和控制信息
	AWREADY	表征写地址已准备完毕
	AWPROT	表征写通道安全标识

图 2-7 为 AXI4 的读传输过程，图 2-8 为 AXI4 的写传输过程。其中读传输用到了 AR 通道和 R 通道，写传输用到了 AW、W 和 B 通道。

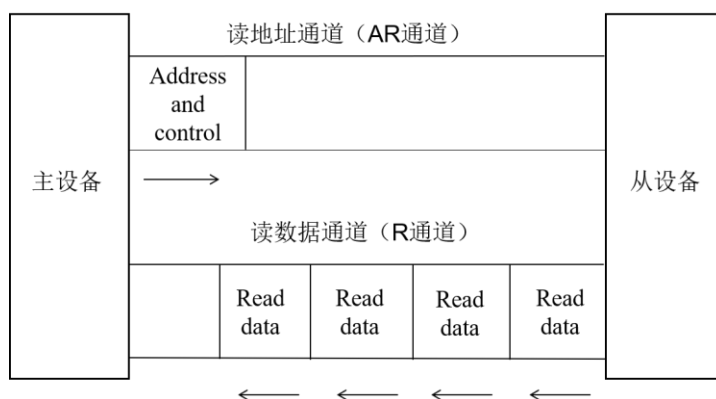


图 2-7 AXI4 读交易过程

如图 2-7 所示，在进行 AXI 通道读数据时，首先使用 AR 通道发送地址和控制消息，然后经过 R 通道传输猝发读数据。

如图 2-8 所示，在 AXI4 读数据传输中，首先经过 AW 通道发送地址和控制消息，然后经过 W 通道传输猝发写数据，最后从设备经过 B 通道回写写响应信号。

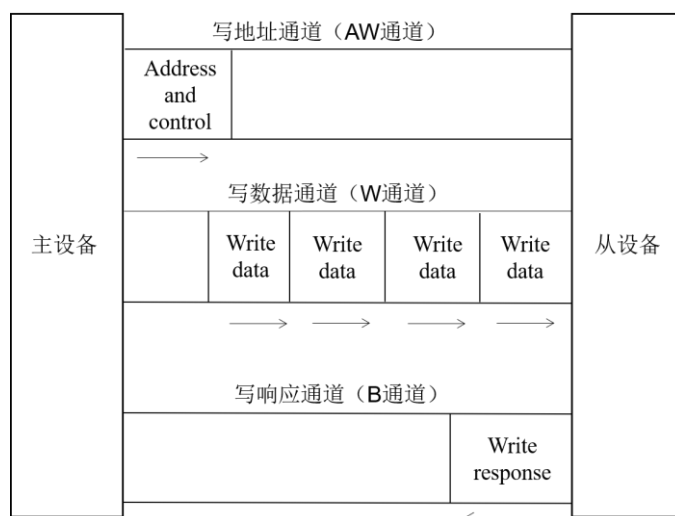
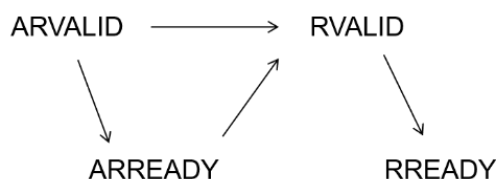


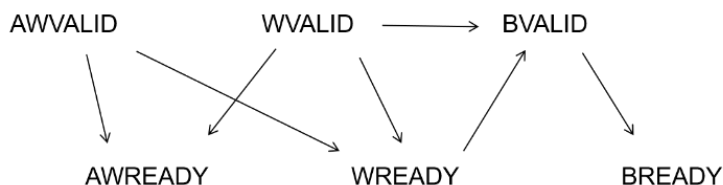
图 2-8 AXI4 写交易过程

2.4.2 AXI 握手机制

AXI 通道主要使用 VALID 和 READY 信号相互配合完成消息收发的握手，消息发送者将 VALID 信号置为高电平表示在当前通道存在有效消息，消息接收者将 READY 信号置为高电平表示已经准备好接收消息。只有这两个信号同时置高时，该通道上的传递的才是有效数据。同时 R 通道和 W 通道还分别存在一个 LAST 信号，此信号用来保存一个完整交易过程的最后一个数据项。以上信号之间有一定的相互依赖，但 VALID 信号不依赖于传输过程中其他通道的 READY 信号。



(a)读过程信号依赖图



(b)写过程信号依赖图

图 2-9 AXI 读写过程信号依赖图

图 2-9 (a) 描述的是读传输过程中信号之间的依赖关系，从设备先等待 ARVALID 信号，然后对 ARREADY 信号进行确认，同时需要等待 ARVALID 信号和 ARREADY 信号到达后才可写 RVALID 信号。

图 2-9 (b) 描述的是写传输过程中信号之间的依赖关系，主设备可以不用等待从设备的 AWREADY 或 WREADY 信号，直接将 AWVALID 和 WVALID 信号置高。而从设备可以等待 AWVALID 信号和（或）WVALID 信号，不需要等待主设备置高 BREADY 信号，然后确认 WREADY 信号。

2.4.3 Zynq AXI 接口

Zynq-7000 SoC 内的 AXI 总线实现 CPU 和 PL 部分逻辑电路的连接，AXI 协议使用主从互联模式实现通信双方之间地址、数据和响应事务通信。值得一提的是，

AXI Interconnect IP 核内部能完成主从设备之间不同 AXI 协议、数据位宽和时钟域的匹配和转换，极大的简化了设备之间的连接结构。

Zynq-7000 芯片内部的 PS-PL 互联结构如图 2-10 所示。

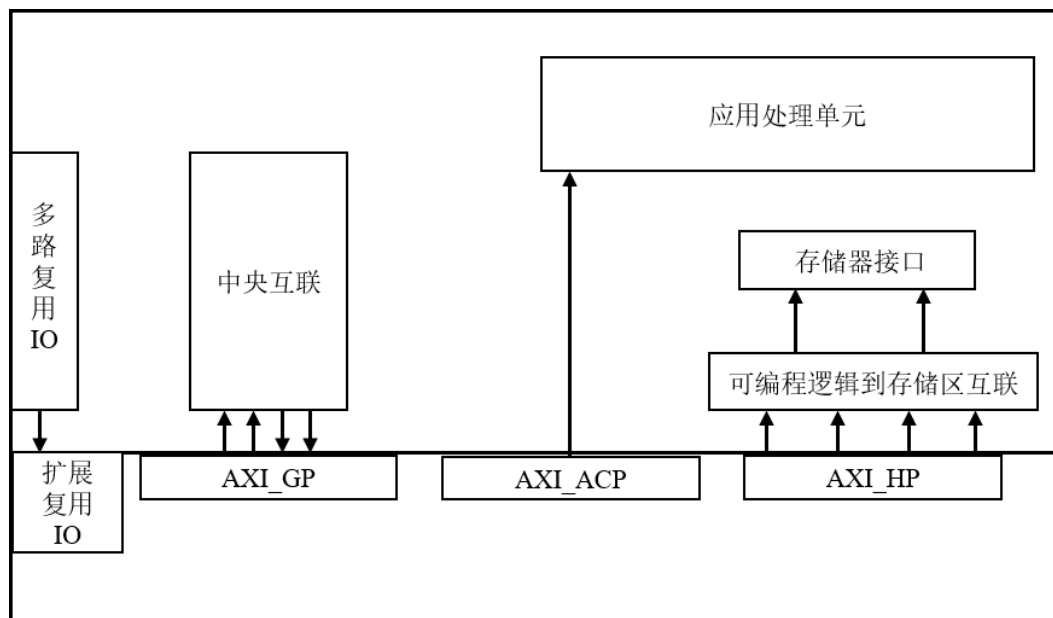


图 2-10 PS-PL 互联结构

Zynq-7000 Soc 中有三种 AXI 接口，分别是 AXI_HP、AXI_GP 和 AXI_ACP 接口，这些接口将 Zynq-7000 SoC 中的 PL 部分连接到 PS 端的各种硬件资源上，完成 CPU 对 PL 端内部逻辑控制以及 PS 与 PL 的信息传输。表 2-4 中包含了接口名称、接口简要描述和该接口的主从设备，其中接口名称中的首字母表示 master 或 slave，其含义分别为此接口对于 PS 而言是主接口和从接口。

AXI 接口连接情况和属性是本文设计需要关注的关键信息，下面具体介绍三种 Zynq 中的 AXI 接口。

1. AXI_HP 接口是 AXI3 规范下的高性能 AXI 接口，Zynq-7000 Soc 中共有四个 AXI_HP 接口，接口两端分别连接 PL 和 DDR。同时每个接口配置了读 FIFO 缓冲和写 FIFO 缓冲。
2. AXI_GP 接口是 AXI3 规范下的通用 AXI 接口，同样在 Zynq-7000 Soc 中有四个，其中两个将 PL 端作为主设备，另外两个将 PS 作为主设备。
3. Zynq-7000 Soc 中还包含一个 64 位的 AXI_ACP 接口，将 PL 端设备连接到 PS 端的 CPU 上，实现了对 PL 端的高速访问且保证二者之间的两级缓存数据一致。

表 2-4 AXI 互联接口

接口名称	接口描述	主设备	从设备
S_AXI_GP0	中吞吐量接口	PL	PS
S_AXI_GP1		PL	PS
S_AXI_ACP0	高速访问接口， 两级缓存的数据一致	PL	PS
S_AXI_HP0	高吞吐量接口 包含读/写 FIFO、两个 DDR 控制器的专用 存储器端口和连接到 OCM 的高性能端口	PL	PS
S_AXI_HP1		PL	PS
S_AXI_HP2		PL	PS
S_AXI_HP3		PL	PS
M_AXI_GP0	中吞吐量接口	PS	PL
M_AXI_GP1		PS	PL

2.5 FC 相关协议分析

如前所述，在机载航电网络和地面仿真系统中，FC 因其低延迟、高带宽、高速率和较强的抗干扰能力得到广泛应用。由于 FC 被越来越多的运用在航空电子系统中，产生了一系列针对于航电环境的 FC 协议，即 FC-AE 协议族，其中本文涉及的 FC-AE-ASM 协议是目前最常用的航电光纤协议之一。本节首先介绍 FC 协议，重点分析协议的帧头结构和层次结构等，然后介绍本文涉及到的子协议 FC-AE-ASM。

2.5.1 FC 协议概述

早在 1994 年 10 月，ANSI（American National Standards Institute，美国国家标准学会）的 X3TXX 小组制定并发布了 FC 协议规范，最初主要被广泛应用于在存储区域网络中，可以很好的解决该网络中使用的并行总线时的一些弊端。为了使光纤通道总线结构更广泛的应用到具体系统中，目前的 FC 协议规范中提供多种节点连接结构的支持，其中最常见的包括点对点结构、包含交换设备的交换式连接结构和仲裁式环形结构，一般在系统中交换式结构使用支持具体协议的 FC 网络交换机来连接各个节点，环形结构和点对点结构则直接连接各个节点，具体结构连接情况说明见图 2-11。

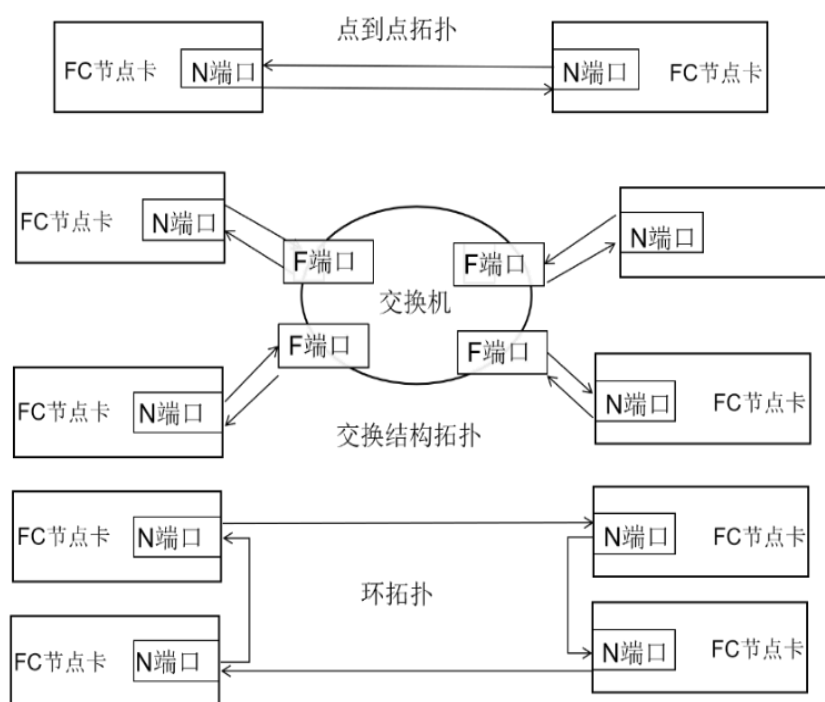


图 2-11 FC 网络支持的三种拓扑结构

2.5.1.1 FC 协议层次

FC 协议中共定义了五个层次结构，每个层次合理分工，各层的名称和简要介绍见图 2-12。

FC 协议规范中对每个层次做了详细的定义和解释，由于内容较多且本文篇幅有限，这里仅简要介绍：

1. FC-0 层：FC 协议中的物理接口层，规范了 FC 接口的物理特性和 FC 中传输介质标准和规范等。
2. FC-1 层：FC 协议中的传输协议层，该层定义了比特流中 DC 均衡规则和 8B/10B 编码规则，为数据流编码提供支持，同时为 FC 设备端口的管理和传输链路路上的控制等功能提供支持。
3. FC-2 层：FC 协议中的信令协议层，该层规范了数据的传输规则和传输机制，实现了数据重组纠错和数据统计等功能。
4. FC-3 层：FC 协议中公共服务层，该层提供数据加密、数据压缩、组查询、多播和分段处理等服务。
5. FC-4 层：FC 协议中的协议映射层，主要实现了 FC 与上层应用的 API，可支持多种上层应用。

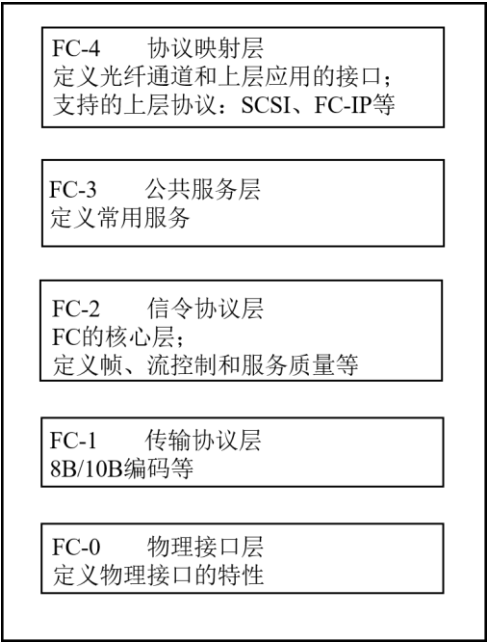


图 2-12 FC 协议层次结构

2.5.1.2 FC 协议帧格式

FC 网络中数据的收发都是以 FC 帧为基本单位的，FC 帧格式在 FC-2 层进行规范，一个完整的 FC 帧格式见图 2-13。

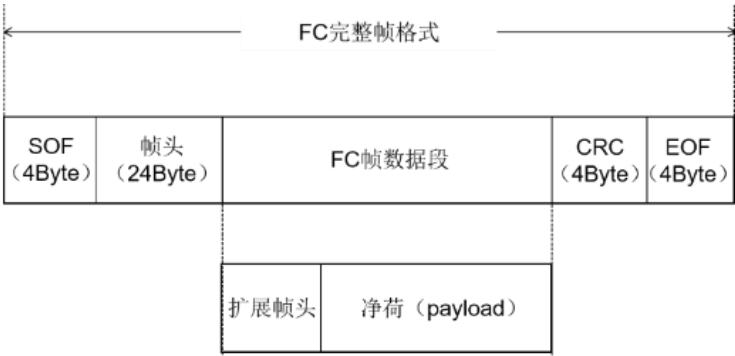


图 2-13 FC 帧格式

一个完整的 FC 帧由 SOF（Start of Frame）、帧头、数据段、CRC 校验字段和 EOF（End of Frame）组成，其中 SOF 和 EOF 分别为帧起始和帧结束界定符，数据段由可选的扩展帧头和 payload 字段构成，扩展帧头是由一些 FC 子协议定义的，其长度不固定，比如 FC-AE-ASM 帧的扩展帧头为 16 字节，而 FC-AE-1553 帧的扩展帧头为 24 字节。Payload 字段为用户设置传输的有效数据，规定数据长度的范围是 0 到 2112 字节，其他字段所占长度是固定不变的，在图 2-13 有所标识。

CRC 为校验字段，位于数据段后面，用于对帧头和 FC 帧数据段的完整性与正确性进行校验。

FC 协议帧头的格式是固定的，负责携带每个 FC 帧的一些必要属性和信息，其具体格式见图 2-14。

位 字 (4字节)	23-16	15-8	31-24	7-0
0	R_CTL	D_ID		
1	CS_CTL	S_ID		
2	TYPE	F_CTL		
3	SEQ_ID	DF_CTL	SEQ_CNT	
4	OX_ID		RX_ID	
5	Parameter			

图 2-14 FC 帧头格式

下面对 FC 帧头的字段进行简单介绍：

R_CTL：路由控制字段，该字段占 8bit，前 4bit 是路由字段，标识源到目的的路径信息；后 4bit 是信息字段。不同的 FC 子协议在该字段的值都有差异，其中 FC-AE-ASM 协议帧该字段对应的值为 0x04。

D_ID, S_ID：目的和源地址标识字段，分别标识帧的目的端和源端。

CS_CTL：特殊控制字段，存放特殊类型的控制信息。

TYPE：帧类型字段，表示帧协议的类型。其中，FC-AE-ASM 为 0x49。

F_CTL：帧控制字段，此字段定义了 FC 帧的控制信息。

SEQ_ID：序列标识符字段，此字段由序列起始端分配，唯一标识 FC 帧的序列，可以识别同一序列交换中不同序列。

DF_CTL：数据段控制字段，用于指示数据字段是否包含扩展帧头。

SEQ_CNT：序列统计字段，用来识别序列中的不同帧，确保接收到的是正确的 FC 帧。

OX_ID, RX_ID：分别为发送端和响应端交换标识字段。

Parameter：参数字段，在数据帧中表示帧在序列中的相对偏移量，在链路控制帧中表示帧信息。

2.5.2 FC-AE-ASM 协议分析

FC-AE-ASM (Anonymous Subscriber Message) 协议是本文涉及的一个 FC 子

协议，专门应用在航电环境中。下面从该协议的特点和帧头结构两个方面进行介绍。

1.FC-AE-ASM 协议的特点

FC-AE-ASM 协议是一种匿名传输协议，消息接收方不需要知道确切的消息发送方，只需要确定每一条 FC-AE-ASM 消息都是来自于单一序列的单项交换。所以在实际的应用中，所有的信息应该使用未经请求的信息类型^[26]。该协议规定同一个帧序列中帧的发生端必须相同，即源地址相同。

2.FC-AE-ASM 协议帧头结构

FC-AE-ASM 通过扩展帧头进行标识，除了包含通用 FC 帧头，FC-AE-ASM 帧还包括了 ASM 扩展帧头，占用图 2-14 中数据段的前 16 字节，其具体结构见表 2-5，具体格式见图 2-15。

表 2-5 FC-AE-ASM 帧结构

SOF (4 字节)	FC 帧头 (24 字节)	ASM 扩展帧头 (16 字节)	载荷 (0-2096 字节)	CRC (4 字节)	EOF (4 字节)
位 字 (4字节)	31-24		23-16	15-8	7-0
	Message ID				
	保留安全字段(Security)				
	保留字段(Reserved)				
	L (1位) + 优先级 (Priority) (7位)	负载长度字段 (Message Payload Length)			

图 2-15 FC-AE-ASM 扩展帧头格式

ASM 帧头中的保留字段 (Reserved) 目前尚未使用，MessageID 字段 (4 字节) 表示发送方发送的消息类型。L 字段 (1 位) 用于标识负载长度， 优先级 (7 位) 则标识消息优先级别，优先级可设置为 0-127，数值越大优先级越高。负载长度字段 (3 字节) 标识消息负载长度，不包括帧头部分的长度。

FC-AE-ASM 帧有两种不同类型的帧序列，分别是块式帧序列和流式帧序列。块式帧序列指的是源地址不同的单帧序列，流式帧序列指的是源地址相同的帧构成的多帧序列。

2.6 本章小结

通过前两节可以对实时系统和实时操作系统有一定的认识，且明确了实时操作系统的性能指标，为实时化改造提供了依据。然后本章详细的介绍了 Xilinx Zynq 平台，主要是 Zynq 平台的总体结构、开发环境和开发流程等。由于 Zynq-7000 SoC 中 PS 和 PL 是通过片内 AXI 总线和 DMA 技术进行数据交互的，本章也详细介绍了 AXI 三种接口类型、AXI 握手机制和 Zynq 平台上 AXI 接口等关键点，对于 DMA 原理也有所提及。最后对于 FC 网络中通用的 FC 协议和本文涉及的 FC-AE-ASM 协议进行了详细分析。

第三章 Linux 实时性分析与实时化改造

Linux 内核发展到现在早已羽翼丰满，在各种性能上的进步也是有目共睹的，这其中就包括实时性能。Linux2.6 系列版本的变化是 Linux 最成功变革之一，从这个版本起支持了内核级的部分抢占，增加了表现出色的完全公平调度策略作为默认策略，使得其达到了软实时操作系统的标准。但是从实时系统应用的角度上看，其实时性还达不到实际的需求。本章是在详细分析了 Linux 内核的实时局限性后，提出一种基于 Zynq-7000 SoC 的 Linux 系统实时性改进方案——Preempt-RT patch。最后针对目标硬件平台，对 Preempt-RT 进行配置与实时平台搭建。

3.1 Linux 实时性研究

3.1.1 Linux 系统概述

Linux 之所以不被认为是 Unix 架构的内核，主要是因为其在设计之初只是借鉴了 Unix 的部分设计思想，没有像其他 Unix 系统一样直接使用 Unix 架构。由于其遵循自由软件协议，吸引了世界上很多工程师对其不断完善^[27]。在嵌入式系统领域中，由于 Linux 系统的内核精简并易于定制和裁剪，相比于其他昂贵的商业实时操作系统来说是更为适合的选择。同时 Linux 可以移植在多种嵌入式系统上，兼容性极强。Linux 支持的详细架构或处理器可以在内核源码的 arch 目录下查看。如图 3-1 所示，arch 的各个目录下存放着 Linux 支持的不同硬件平台的相关代码。

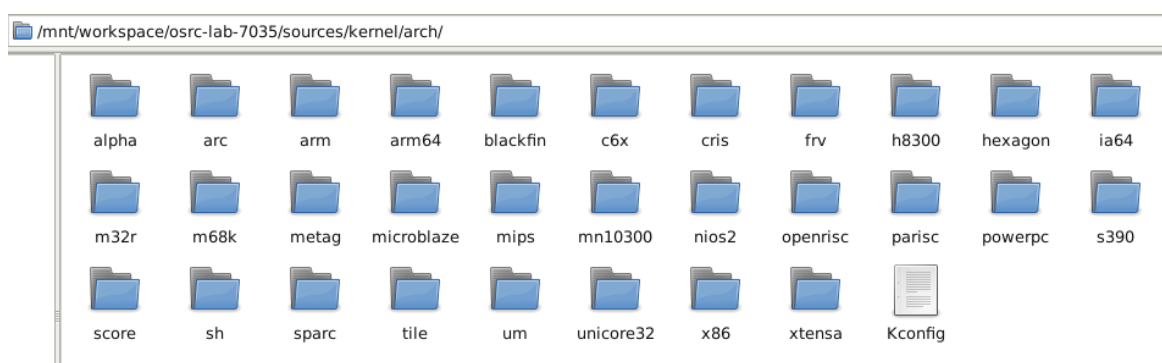


图 3-1 arch 目录结构

GNU/Linux 系统的一个通用的系统架构如图 3-2 所示。

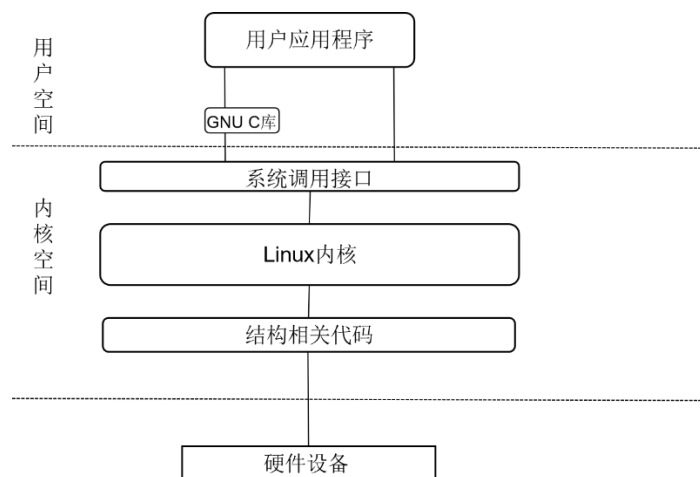


图 3-2 GNU/Linux 系统架构

Linux 内核按照其功能可以划分成几个分系统，一是进程管理，主要功能是将系统中的任务抽象为相对独立的进程，便于进行多任务调度，提高并行运行效率；二是网络通信管理，该部分实现了系统支持的网络协议，同时给应用层提供网络通信的接口；三是内存管理，与内部存储资源相关的代码，其中用到了虚拟内存和分页分段等管理手段，实现了内存的合理分配和回收；四是虚拟文件系统，该部分对系统中任何相对独立的信息体甚至硬件设备进行抽象，同时给应用层提供文件操作和文件系统相应接口；五是硬件设备管理，给所有连接到平台上的设备进行分类，同时可以以模块的形式加载驱动设备的代码。内核在设计的过程中，既要考虑兼容多种硬件平台，又要考虑每个硬件平台的板级关系，使得内核中有大量和硬件结构相关的代码。同时还提供了大量系统调用 API 给用户空间，既隔离了内核空间代码，保证内核数据的安全性，又能让应用程序便捷的使用内核功能。

3.1.2 Linux 的实时性限制

Linux 从诞生到目前的发展，在进程调度方面尽量实现所有进程（或线程）公平调度，使得相同优先级的进程得到的 CPU 时间也相同，同时努力实现其他硬件资源在进程之间公平共享；在性能方面将实现最大吞吐量作为首要目标，故实时性能并没有得到太多关注。从 2.6 系列版本之后，系统实时性逐渐得到重视，但是前提是不明显影响 Linux 的其他性能。这也使得标准 Linux 内核在实时性能上仍然受到一些因素的影响，下面进行详细分析。

1. 时钟粒度

操作系统得以在正确的轨道上运行，离不开系统中的时钟管理体系。时钟粒度可以简单解释为系统能分辨的最小时刻，也是整个系统最小的工作节拍^[28]。如果

时钟粒度过大,会影响操作系统对关键性任务的调度,使得调度延迟过大而超过任务最晚截止时间。为保证性能均衡在其时钟频率配置时一般配置 100Hz,这种粒度对于很多关键任务来说是非常粗糙的。

2. 内核的抢占性

较早的 Linux 内核版本不支持在内核空间抢占执行权,当一个实时任务准备完毕并等待执行时,如果当前非实时任务正在内核空间运行,比如调用系统调用陷入内核。由于内核不可抢占,实时任务得不到及时执行,这势必会影响实时性。关注到此问题后,目前的 Linux 内核版本已经具有了非常成熟的内核抢占机制,使得一些非关键任务进入内核空间执行时也可被优先级更高的关键任务抢占 CPU。但是为了保证内核的其他性能,抢占点的增加还是有所保守,这使得在大量情况下仍然禁止优先级抢占。比如当前 CPU 正在运行一个非关键任务并陷入内核,同时获得自旋锁(spin_lock)进入临界区,此时一个实时任务被唤醒,但是由于自旋锁不可被抢占,此时系统会锁住抢占调度,无法确定实时任务得到调度的具体时间^[29]。

3. 中断机制

Linux 系统不支持硬实时的另一个重要原因是由内核的中断机制导致的。Linux 在响应中断时会执行其对应的中断处理程序,一般而言,中断处理程序应该在较短的时间内执行完毕,但是对于复杂的中断来说,又需要中断处理程序执行较多的耗时的的工作,这使得两个诉求相互矛盾,所以 Linux 一般把中断区分为两个部分:上半部和下半部。接收到对应中断后,上半部即刻开始关中断执行,此过程中会禁止一切调度,包括实时任务的调度。能够允许被延后执行的中断任务会被放在中断处理程序的下半部,此部分执行时可以被其他中断打断^[30]。当系统中的中断频繁出现,特别是中断服务程序的上下部划分不合理时,Linux 的关中断机制也会对实时性有较大的影响。

3.2 Linux 的实时化改造方案

从内核源码设计的角度分析 Linux 系统的实时性,发现有很多限制因素存在,为了满足应用的需求,需要对内核进行优化以提高实时性。目前主流的 Linux 实时优化按照其实现方式可分为双内核方案和修改内核源码方案两种^[31]。

3.2.1 双内核方案

这种方案的具体做法是在一个硬件平台上运行两个操作系统内核,以 Linux 内核为基础并行运行一个硬实时的微内核,彼此合理分工:实时微内核为系统中的关键任务提供专门的调度支持,Linux 内核则完成其他非实时的通用功能^[32]。这种改

造策略实质上是在关键性任务上完全放弃了 Linux 内核，而选择具有硬实时特性的精简内核，该内核将 Linux 看作非实时进程处理。目前，双内核改造的两个成熟的方案是 RTAI 和 Xenomai^[33]。两种方案的实质是相同的，都是基于操作系统自适应域环境（Adaptive Domain Environment for Operating System, ADEOS）机制实现的，ADEOS 依附于标准 Linux 内核，可以以 patch 的形式加入内核执行^[34]。两种双内核方案的架构对比见图 3-3，细微的不同之处在于 RTAI 作为实时内核会将一些不重要的实时任务通过直接操作硬件的方式传递给内核，以此提高实时性。

ADEOS 的运行机制包括域管理机制和中断管道机制^[35]。域管理机制把每个操作系统内核当作一个域进行管理，每个域有自己的优先级和地址空间等资源。中断管道机制提供了一种让中断在每个域之间传递的方法，优先级高的域首先处理中断，然后再决定是否终止传递。值得一提的是，在 ADEOS 出现之前，双内核的实现依赖于一个实时硬件抽象层，众多实践证明，ADEOS 可以使多个操作系统的配合更加协调且运行更加稳定。

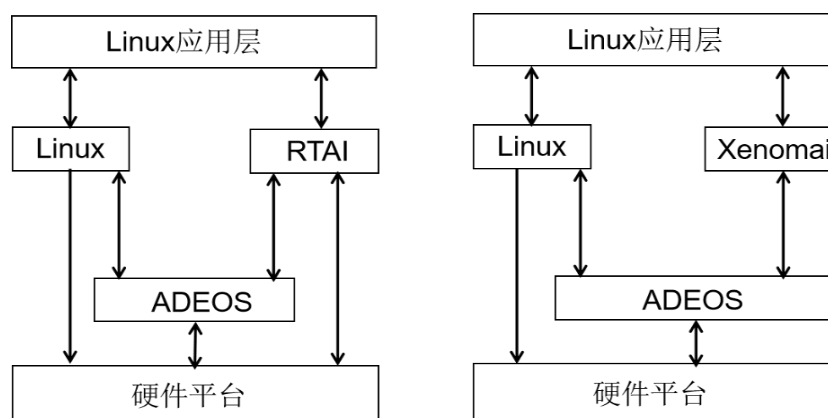


图 3-3 RTAI 与 Xenomai 方案架构对比

3.2.2 修改内核源码方案

在实际的工程中，修改 Linux 内核源码是更常用的一种 Linux 实时性优化的方法，这种方案的活跃得力于 Linux 内核源码的开放特性，使得大量开发人员开发出各种出色的实时补丁（patch）^[36]。直接修改内核源码方案和双内核法相比，最大的优势是它不改变 Linux 系统的系统调用接口，用户在开发应用和驱动程序时基本感觉不到与普通 Linux 内核之间的差别。在本方案中应用最广泛的一个实时抢占补丁是 Preempt-RT patch，其核心思想是最大化内核空间中的允许抢占点，使得优先级较高的关键任务在更多的地方可以抢占当前任务的 CPU 时间获得执行。同

时也使内核满足了一些重要的实时特性（将在 3.3 节介绍），可以显著降低系统中任务响应延迟的抖动，且将延迟控制在一个较小的范围内。

3.3 Preempt-RT 的关键技术研究

由于 Preempt-RT 一直把加入 Linux 主线作为发展目标之一，所以在提高 Linux 实时性的同时，力求对标准 Linux 的其他性能影响降到最低。下面将详细研究 Preempt-RT 实时优化 Linux 的三个关键点。

3.3.1 优先级继承

Preempt-RT 的一大亮点是提出并实现了进程优先级继承协议，很好的解决了标准 Linux 内核中不可预期的优先级反转问题^[37]。首先通过一个例子说明一种优先级反转的情况，如图 3-4 所示，当前有 P1、P2、P3 三个进程，他们的优先级大小对比为 $P1 < P2 < P3$ ，其中 P1 和 P3 都有一段临界区。三个进程就绪时刻分别为图 3-4 中 t_1 、 t_2 、 t_3 。图 3-5 描述的就是一种优先级反转的情况，假设 P1 和 P3 需要进入同一个临界区。首先优先级最低的进程 P1 运行，成功进入临界区，此时进程 P2 到达，由于 P2 的优先级较高，所以 P2 抢占 P1 获得 CPU 开始执行。P2 执行到 t_3 时，优先级更高的 P3 到达抢占 P2 获得 CPU 执行，执行到临界区时进入失败变成阻塞状态，此时 P2 获得 CPU 运行直到结束，然后 P1 运行直到释放互斥锁后 P3 才得以继续运行。此时进程 P2 和 P3 发生了优先级反转，即 P3 的优先级大于 P2 却晚于 P2 执行完毕。

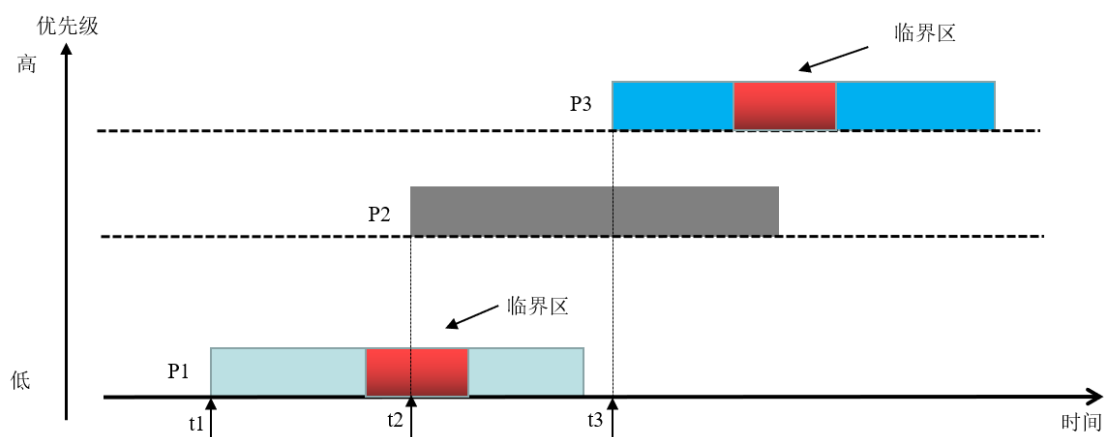


图 3-4 进程优先级以及到达时刻示意

优先级继承指的是当进程间由于抢占资源发生优先级反转时，临时提高优先级较低进程的优先级，使其值等于优先级较高进程的优先级，直到其占用的资源被释放^[38]。故上面的例子中，当高优先级进程 P3 等待进入临界区时，进程 P1 则继承 P3

`spin_lock`, 替代后 `rt_mutex` 所保护的临界区可被无竞争的更高优先级的任务抢占。对于 Linux 内核中的中断处理见图 3-7, 在时间 a 处产生一个中断, 由于此时处在无可抢占点的临界区, 所以中断在临界区结束点 b 处得到响应, 执行中断处理程序唤醒关键任务, 中断处理延迟的时间为 $b-a$ 。如图 3-8 所示, 对于 Preempt-RT 改造的实时 Linux 内核来说, 临界区内增加了很多抢占点, 此时关键任务会及时得到执行, 中断处理延迟的时间为 $c-a$, 明显降低延迟时间。

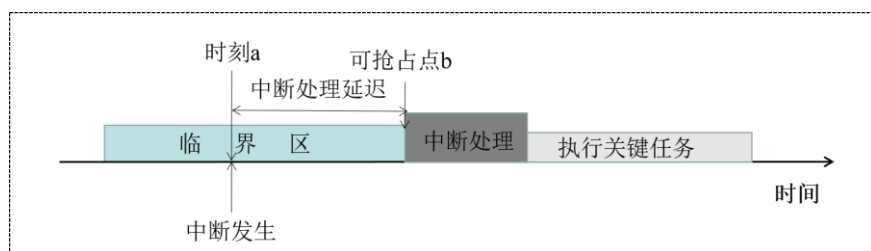


图 3-7 标准 Linux 内核中的中断处理过程

需要注意的是, 并不是所有的不可抢占锁都可以被 `rt-mutex` 替换, 对于中断处理程序中的临界区、内核中一些特殊的数据结构和一些重要的寄存器不可被抢占。因为中断处理程序是不可被重新调度的 (线程化的中断处理程序下半部分除外), 如果其在临界区中因被抢占而陷入睡眠, 则该临界区资源将会永远得不到释放, 很大几率会造成系统僵死。同样地, 如果对一些重要的寄存器或内核关键数据结构读写时因被抢占而中断, 也可能产生严重后果。

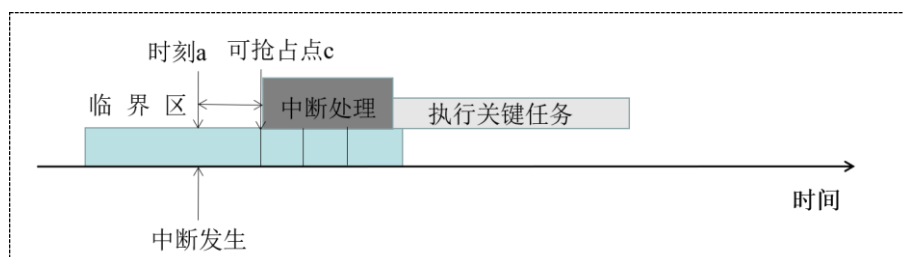


图 3-8 实时 Linux 内核中的中断处理过程

3.3.3 中断线程化

标准 Linux 内核对硬件中断处理时, 为了防止出现级联中断问题, 需要关闭中断, 会阻断其他的中断请求。所以在执行中断处理程序的过程中, 任何任务都得不到调度。由于中断在何时产生是无法预测的, 因此实时任务得到调度的时间也不能确定。同时如果频繁产生外部 I/O 中断, 此时就绪的关键任务会一直得不到执行。Preempt-RT 为了解决这个问题, 给内核引入了中断线程化的实时特性, 强制将通

过 `request_irq()` 函数申请的中断的处理程序下半部在内核线程中开中断执行，使得中断下半部分在内核线程的上下文中可以被抢占和调度。值得关注的是，虽然中断线程化几乎可以无差错的优化系统中任何中断，但是也有例外。比如最常用的时钟中断，它为 CPU 和整个系统工作提供了基本的时间基准，是整个系统正确工作的前提，所以不能被线程化。

3.4 Preempt-RT 的配置与移植

3.4.1 交叉编译环境的搭建

因为嵌入式系统资源受限，在嵌入式平台上安装编译工具进行程序的开发和编译将会非常困难，而且嵌入式处理器的运算能力不足，直接在嵌入式平台上编译程序将会大量增加耗时，过程中会经常由于各种原因编译中断或失败。所以对于嵌入式设备的开发通常采用宿主机-目标机设计方法，宿主机一般是装有 Linux 系统的 PC (x86 架构)，目标机就是需要开发的嵌入式设备^[40]。在宿主机上搭建交叉编译环境，然后进行软件开发后，利用交叉编译工具编译成可以在目标平台上运行的可执行程序，最后烧写到目标机上运行测试^[41]。

本工程的目标平台 MZ7035 在 2.3 节已经介绍过，它是在 Zynq-7000 SoC 系列芯片 XC7Z035 的基础上，同时增加了一些必要的外设构成的 Zynq 开发平台。

交叉编译工具链是编译工具的集合，其中主要的组成要素见表 3-1。

表 3-1 交叉编译工具链的组成要素

组成部分	源码包	说明
编译器 gcc	gcc	工具链中最核心、最常用的三个工具； 工具名一般以“arm-linux-”开头；
汇编器 as	binutils	
链接器 ld	binutils	
内核头文件包	linux 源码	通过包含 Linux 内核头文件调用系统调用；
C 函数库	GNU/libc, uClibc, newlib	GNU/libc 是标准 C 函数库； uClibc 和 newlib 是嵌入式系统轻量级库
其他工具集	binutils	常用的包括 nm、size、objcopy、objdump、strip 等

交叉编译工具链可以在网上下载适用于 ARM 处理器的通用安装包，下载时选择好对应目标架构的具体版本。同时在 Linux 版本 Vivado 套件中的 SDK 下，也集成了完整的交叉编译工具链，可以在开发过程中选择使用。

3.4.2 Preempt-RT 的配置与移植测试

本文所使用的 Linux 内核是 Xilinx 提供的 4.14.0 版本的,通过和 Linux 官方的内核源码比较发现,内核的核心功能实现部分是完全一致的,故理论上移植 Linux 官方发布的 Preempt-RT 不会出现一些兼容问题,之后的工程也印证了这一点。Preempt-RT patch 版本是随着内核版本一起更新的,可以在 Linux 官网上找到,如图 3-9 所示。为了避免内核与 Preempt-RT patch 同时工作时出现不可预期的错误,Preempt-RT patch 的版本应该和 Linux 版本完全一致。

2.6.25/	08-Aug-2013 18:27
2.6.26/	08-Aug-2013 18:28
2.6.29/	08-Aug-2013 18:28
2.6.31/	04-Nov-2014 14:19
2.6.33/	08-Aug-2013 18:29
3.0/	19-Nov-2013 22:02
3.10/	23-Nov-2017 05:44
3.12/	08-Jun-2017 13:40
3.14/	13-Feb-2017 22:26
3.18/	23-May-2019 16:23
3.2/	23-Nov-2017 05:53
3.4/	16-Nov-2016 19:26
3.6/	19-Nov-2013 22:01
3.8/	04-Nov-2014 13:35
4.0/	13-Jul-2015 21:06
4.1/	29-Nov-2017 22:12
4.11/	17-Oct-2017 13:42
4.13/	17-Nov-2017 17:03
4.14/	12-Jan-2021 16:45
4.16/	03-Aug-2018 07:39
4.18/	29-Oct-2018 11:51
4.19/	08-Jan-2021 18:17
4.4/	04-Jan-2021 10:15
4.6/	30-Sep-2016 21:37
4.8/	23-Dec-2016 15:26
4.9/	12-Jan-2021 20:00
5.0/	10-Jul-2019 15:17
5.10/	08-Jan-2021 19:24
5.2/	16-Dec-2019 17:11
5.4/	08-Jan-2021 16:55
5.6/	20-Aug-2020 14:23
5.9/	28-Oct-2020 20:05

图 3-9 Linux 不同版本的补丁目录

将下载到的 patch-4.14-rt1.patch.gz 文件拷贝到 Linux 内核源码的同一目录下,在该目录下使用 zact 命令将 Preempt-RT patch 加入到 Linux 源码中,打好补丁后可以对带有 Preempt-RT patch 的 Linux 内核进行图形化配置。过程中需要注意的是,配置项 Preempt Model 一栏应选择 Fully Preemptive Kernel(RT)选项以开启实时抢占,否则 Preempt-RT 不会介入 Linux 内核运行, Fully Preemptive Kernel(RT)选项配置见图 3-10。

配置完成后,就可以交叉编译内核产生 uImage 镜像文件。对于此实时内核,可以进行启动测试,准备好启动必需的 BOOT.bin、设备树文件和根文件系统,放入 SD 卡对应的分区,调整 Zynq 平台开发板的启动选择开关,选择使用 SD 卡启

动。打开串口，就可以看到实时 Linux 系统的启动信息，见图 3-11。对其实时性能的测试将在 5.1 节介绍。

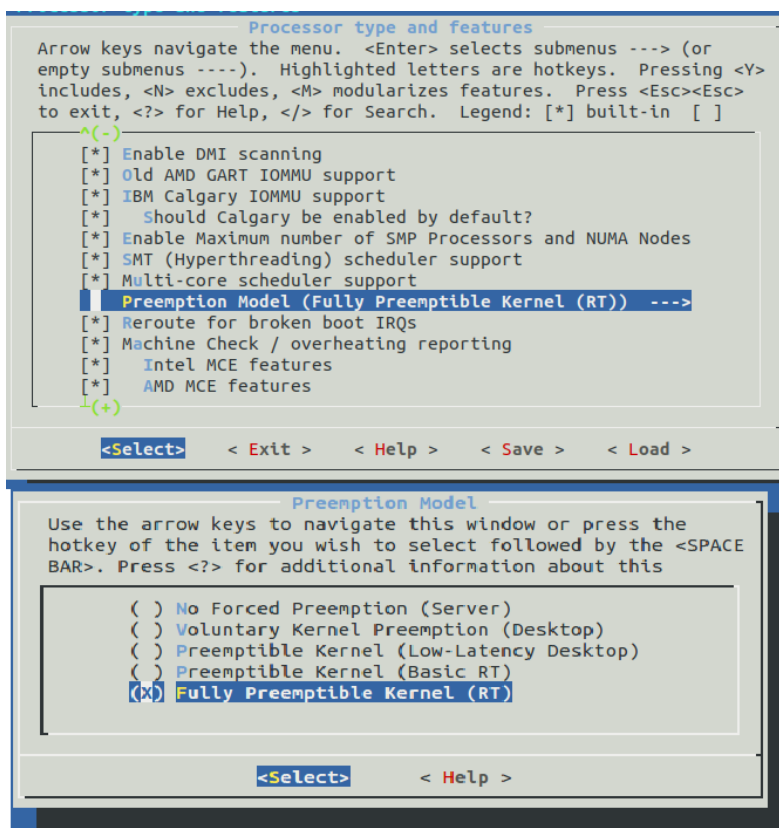


图 3-10 Fully Preemptive Kernel(RT)选项配置界面

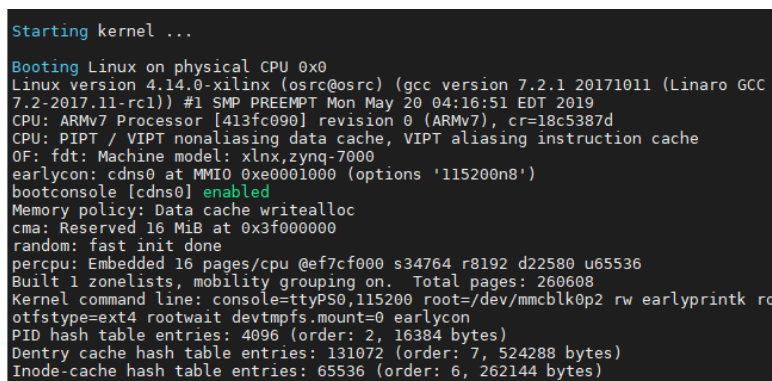


图 3-11 Linux 启动界面

3.5 本章小结

本章首先研究 Linux 内核的实时特性，详细提出一些主要的实时性限制因素；并针对目前 Linux 内核的实时性不足问题提出两种改进的方案，根据本设计的工

程需求, 最终选择直接修改内核法对 Linux 内核进行改造。接着研究了 Preempt-RT 补丁的关键技术, 主要介绍了其实时化改造的三大基本措施: 优先级继承、临界区可抢占、中断线程化。最后将 Preempt-RT 进行配置并移植到目标平台 Zynq 上, 进行系统启动测试。

第四章 FC 节点卡软件的设计

在对基于 Zynq-7000 SoC 的 Linux 系统进行实时化改造后,本章重点设计基于 Zynq 实时平台的节点卡软件和软硬件数据交互通道部分。节点卡支持 FC-AE-ASM 协议,一般叫作 FC-AE-ASM 节点卡(下文简称为 FC 节点卡)。首先介绍航电地面仿真系统的构成和工作方式,然后提出 FC 节点卡整体设计架构,最后阐述 FC 节点卡软硬件数据交互通道设计、节点卡驱动程序设计和节点卡应用程序设计。

4.1 航电地面仿真系统的构成

如图 4-1 所示,在航电地面仿真系统中,存在 FC 总控机、FC 交换机、FC 节点卡和转接卡等设备。FC 节点卡负责实现 FC-AE-ASM 帧的生成、帧数据的发送和接收、总控机命令解析等功能,配合 FC 交换机和 FC 总控机搭建基于 FC-AE-ASM 网络的数据传输平台。

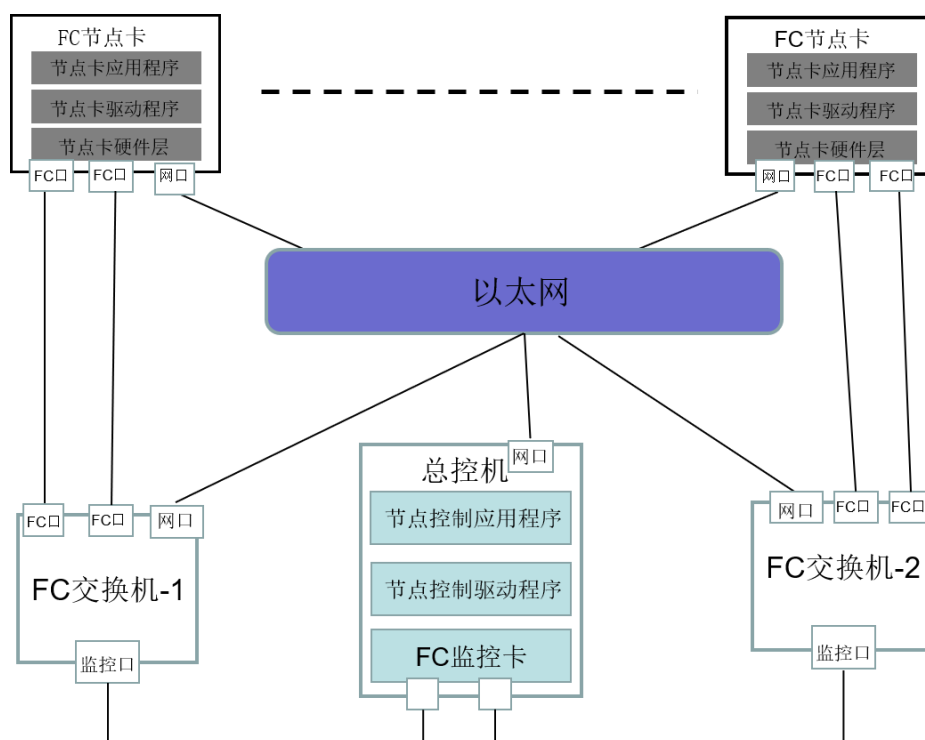


图 4-1 系统总体设计架构

FC 网络中的节点卡一般存在多个,是整个系统中需要通信的节点,这些节点卡或通过交换机连接,或直接连接,统一由总控机控制和管理。总控机是整个航电地面仿真系统的管理中心,它通过以太网与 FC 网络中的节点(节点卡、转接卡、

FC 交换机) 进行远程网络通信, 同时通过可视化界面实现实时显示和人机交互。在总控机上输入连接到网络中的节点卡 IP 地址, 完成 TCP 通信的连接。对于已经建立好连接的节点卡, 总控机可以进行参数配置, 如链路速率切换和端口模式切换等, 同时可以进行 ASM 帧发送配置和调度表生成, 然后将 ASM 帧下发给节点卡进行发送, 在接收方向上, 总控机会获取节点卡的相关参数, 并等待节点卡上传接收到的 ASM 帧, 解析后在界面进行显示。

4.2 FC 节点卡整体架构

基于 Zynq 平台的 FC 节点卡的整体设计架构如图 4-2 所示, 主要由三个部分组成: 底层硬件逻辑、驱动程序和应用程序。底层硬件主要实现 FC 协议结构中的 FC-1 层、FC-2 层、FC-3 层和 FC-4 层的功能, 包括数据的串并转换、并串转换、编解码、数据收发和软硬件数据交互通道等。驱动程序配合 FC-4 层硬件实现帧的封装和传输功能, 同时也给应用程序提供相关接口。上层应用则负责和总控机通过以太网通信, 解析总控机的命令并控制驱动程序执行, 在总控机和驱动程序之间发挥信息传输的作用, 下面给出各层的功能概要。

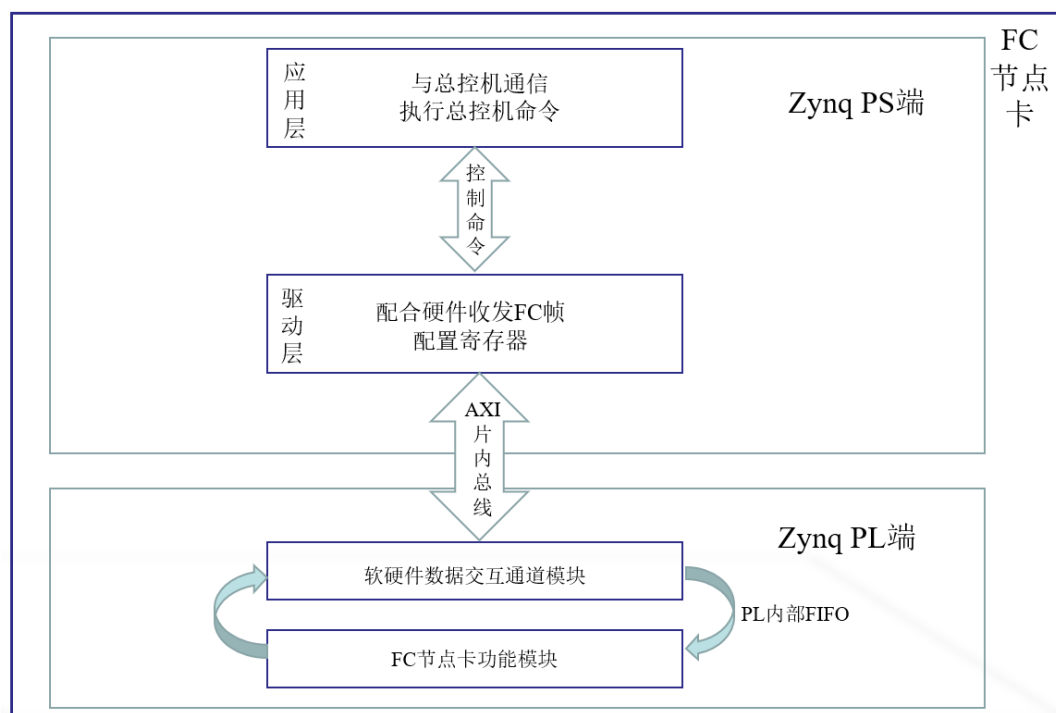


图 4-2 FC 节点卡架构

FC 节点卡硬件层: 该层是由 Zynq 中的 PL 端 (即 FPGA) 完成, 分为两个部分: 数据交互通道 FPGA 设计部分和节点卡功能 FPGA 设计部分。在基于 Zynq 的

FC 节点卡中, 数据交互通道部分需要实现两个功能, 一是要完成 PS 端和 PL 端 FC 帧的交互, 且要达到一定的交互速率; 二是要实现可以由 CPU 进行读写和配置的硬件功能寄存器。节点卡功能 FPGA 设计部分主要完成 FC 帧的收发, 在数据发送方向上从数据交互通道部分的 FIFO 模块获得数据, 然后进行 8B/10B 编码; 最后将并行数据转换为串行数据后经过 FC 接口发送出去。在数据接收方向则相反, 首先对 FC 端口传送过来的数据进行光电转换和串并转换, 然后进行 8B/10B 解码, 解析 FC 帧结构, 最后将解析完的数据发送到数据交互通道部分的 FIFO 中, 由数据交互通道发送给软件部分。鉴于节点卡功能 FPGA 设计部分不是本文的重点工作内容, 故不做详细介绍。

FC 节点卡驱动程序: 在 Linux 驱动程序设计架构下实现, 主要实现的功能包括硬件设备的识别、硬件寄存器初始化、配置和读取寄存器的值、配合硬件设备和应用程序完成收发帧等^[42]。

FC 节点卡应用程序: 节点卡应用程序相当于总控机和驱动程序的中间传输层, 它需要与总控机建立网络通信, 负责获取总控机的命令然后通过一定的机制传递给驱动程序, 同时将从驱动程序获取到的数据返回给监控机。

综上, 本工程的主要设计目标有三个部分, 分别为数据交互通道设计部分、节点卡驱动程序设计部分和节点卡应用程序设计部分。

4.3 数据交互通道方案选择与设计

数据交互通道的设计是在 Zynq 平台的 PL 端进行的, 实现软硬件之间数据的收发和 CPU 对硬件的配置功能, 主要使用到 2.4 节介绍的 AXI 总线和 DMA 传输技术。值得注意的是, 在选择 Zynq-7000 SoC 中软硬件通信方式时, 应当重点考虑 PS 与 PL 之间的传输速率的大小, 以满足对性能的需求^[43]。下面将进行详细介绍。

4.3.1 数据交互通道方案的选择

表 4-1 给出了 Zynq 平台几种不同的数据交互方式。系统吞吐率反映了单一方向上的最大吞吐率, 是首先需要关注的重要指标, 同时软件的设计复杂度和硬件资源占用情况也是需要考虑的因素。

其中使用 CPU 控制的 PS 端 I/O 虽然在软件上是最容易实现的, 而且不占用 PL 端资源, 但是吞吐率太低, 仅适合做一些简单的控制; PS 内的硬核 DMA 控制器 (DMA Controller, DMAC) 虽然能达到比较高的吞吐率, 但是对 DMA 控制都需要使用软件实现, 且实现难度较大, 一般当 PL 端资源紧缺时使用此方式。

AXI_GP 接口的 PL 端 DMA 传输方式的估计吞吐率可以达到 600MB/s, 可以

获得与 PS 端 DMAC 传输方式相同的最大吞吐率^[44]。使用 AXI_HP 接口的 PL 端 DMA 传输方式可以得到更高的吞吐率,最大可达到 1200MB/s。如 2.3.1 小节可知,Zynq-7000 Soc 中共有 4 个 AXI_HP 接口,接口两端分别连接 PL 和 DDR。同时每个接口配置了读 FIFO 缓冲和写 FIFO 缓冲。使用 PL 内的 DMA,虽然需要占用较多的可编程逻辑资源,但是最大吞吐率是使用 PS 内硬核 DMA 的两倍,且软件编程更为简单,一般使用此方式进行 PL 端和 PS 端之间大量数据传输。

表 4-1 Zynq-7000 的数据传输方法比较

方法	资源 占用	软件编程 难度	估计 吞吐率	使用说明
PS 端 I/O	极少	简单	<25MB/s	适用于一些简单的控制
PS 端 DMA	较少	复杂	600MB/s	一般当 PL 端资源受限 或仅进行 PS 端编程时使用
PL 端 AXI_GP 接 口的 DMA	较多	中等	600MB/s	吞吐率中等
PL 端 AXI_HP 接 口的 DMA	较多	中等	1200MB/s	吞吐率较高 适用于大量数据的高速率传 输
PL 端 AXI_AC P 接口的 DMA	较多	中等	1200MB/s	具有高速一致性,延迟较 低,但是接口数量较少

使用 AXI_ACP 接口的 PL 端 DMA 传输方式的吞吐率也可以达到要求,但是 AXI_ACP 接口的主要优点是可以保证各级缓存在数据的一致性,而付出的代价则是占用过多的硬件资源,同时可能会出现系统抖动的问题。系统抖动可理解为由于缓存中数据频繁改变,使得系统会频繁产生缺页错误且无可换页替换,导致必须立即置换在使用页的大量页面调度活动,系统抖动及其带来的并发问题会对整个系统产生较大的负面影响。

由以上讨论可知,使用 AXI_HP 接口和 PL 端 DMA 方式进行 PL 与 PS 之间的数据交互可以使软硬件通信速率达到最大,同时在软件编程和接口性能方面也是比较好的选择。所以本设计选择表 4-1 中的第四种方案实现 PS 端与 PL 端 FC 数据帧的传输。

4.3.2 数据交互通道的设计

数据交互通道部分的设计在 Xilinx Vivado 2017.4 中进行,下面首先给出整体

硬件设计架构，然后对用到的重要的 IP 核的配置和接口连接情况进行详细介绍。

由 4.3.1 小节讨论，采用 AXI_HP 接口与 PL 端 DMA 方案实现 PL 端与 PS 端 FC 数据帧的交互。同时采用 AXI_GP 接口和 Block RAM 模块进行节点卡功能部分的寄存器组的配置，以此实现对硬件的高效控制和信息获取。数据交互通道整体结构见图 4-3，蓝色虚线框内为 Zynq 的 PL 部分，其中 AXIDMA、BRAM 和 FIFO 为 Vivado IDE 中的 IP 核。

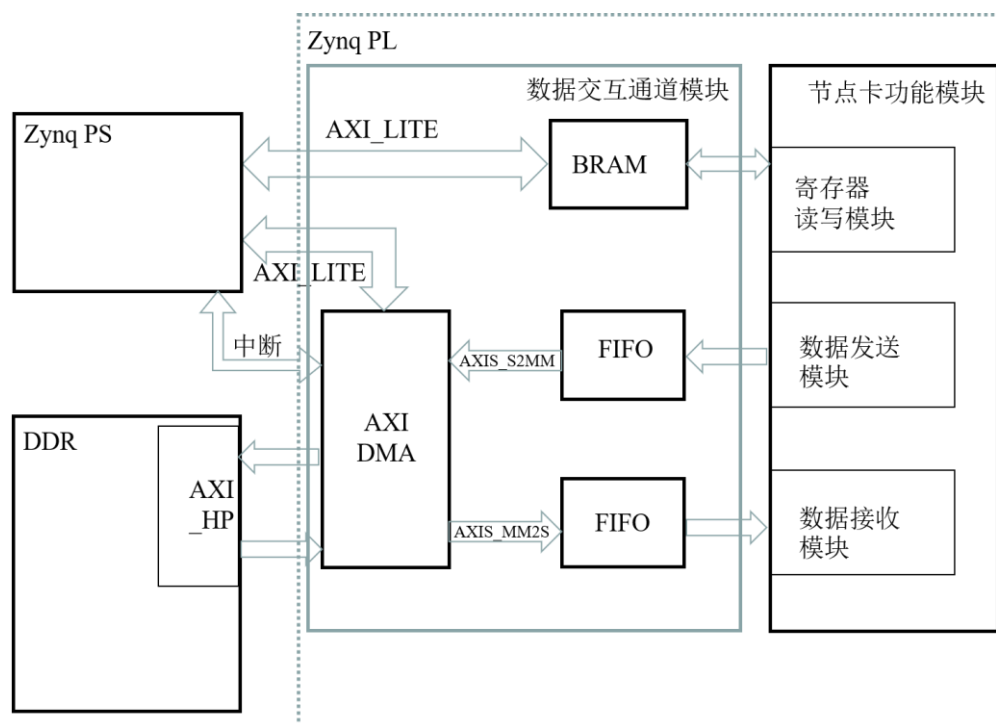


图 4-3 数据交互通道整体结构图

在该部分的 FPGA 设计中，FC 帧从 PS 端的 AXI_HP 口中发出，同时控制消息从 AXI_GP 口经过 AXI_LITE 发出。DMAC 在要接收数据或者发送数据时发出两个不同的中断信号，处理器接收到信号后立即让出总线，由 DMAC 完成 PL 与 DDR 之间的 FC 帧发送和接收。PS 通过 AXI_GP 接口连接到 BRAM 上，对 BRAM 中的寄存器进行配置。同时节点卡功能部分的寄存器读写模块通过访问 BRAM 进行寄存器的读写。FC 帧发送模块将要发送的帧传入 FIFO 中，在经过 AXI_DMA 传入 PS 端的 DDR 中；相反当 DDR 中有数据要发送时，AXI_DMA 将数据传输到 FIFO 中，然后传入 FC 数据接收模块。

其具体的硬件工程实现图如图 4-4 所示，其中使用到的 IP 核包括 Zynq-7000 PS、AXI_DMA、FIFO、AXI BRAM Control、BRAM Generator、PS Reset 和 AXI Interconnect 等。

整个设计中包括如下三种数据通道。

PL 到 PS 数据通道：外部 FC 数据发送模块经过 S_AXIS 接口传入帧数据，再经由 AXI4-Stream Data FIFO_0 模块（数据接收缓冲模块）的 M_AXIS 接口传入 AXI DMA 模块中。数据接收缓冲模块传来的是无地址的流式消息格式数据，在 AXI DMA 模块切换为内存映射格式数据，然后从 M_AXI_S2MM 接口输出，经过 AXI 总线传输到 PS 端的 DDR 中做进一步处理。

PS 到 PL 数据通道：与 PL 到 PS 数据通道相反，数据从 PS 端的内存中通过 AXI_HP0 接口传入 AXI DMA 的 M_AXI_MM2S 中，在 AXI DMA 模块进行地址映射数据到流式数据的转换后，经过 M_AXIS_MM2S 接口传入 FIFO_1 中，最终经过该模块的 M_AXIS 接口传输到 FC 节点卡功能模块进行下一步处理。

寄存器配置通道：由 PS 端经过 M_AXI_GP 接口发出，通过 AXI 总线传入 AXI BRAM Controller 模块中，然后传入双端 BRAM 中的 BRAM_PORTA 接口，对其特定地址的寄存器进行配置。外部节点卡功能模块通过 BRAM 的 BRAM_PORTB 口读取寄存器的值。

下面对设计中用到的几个重要的 IP 核进行配置和连接情况的说明。

4.3.2.1 Zynq-7000 PS 的配置与连接

Zynq-7000 PS IP 核的型号对应于所使用开发板中的 Zynq XC7Z035 芯片，其配置后的 IP 核见图 4-5，Zynq-7000 PS IP 核的接口说明见表 4-2。

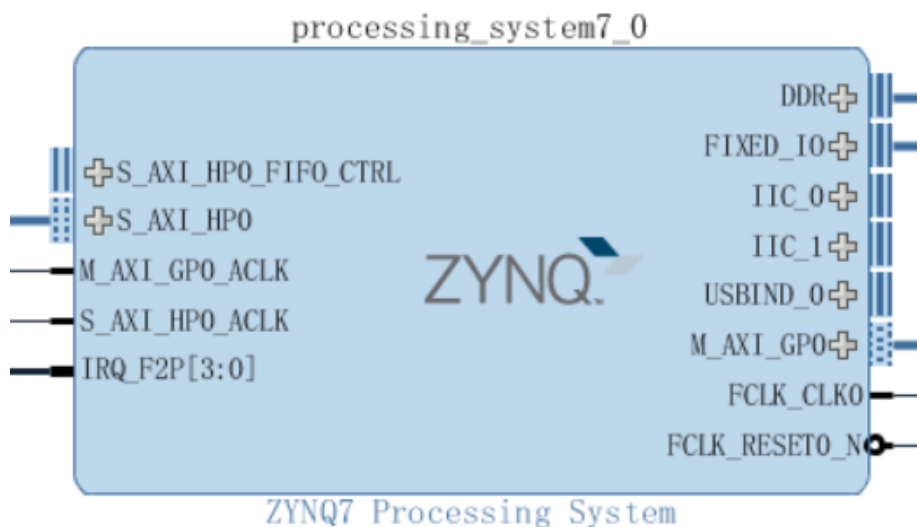


图 4-5 Zynq-7000 PS IP 核

表 4-2 Zynq-7000 PS IP 核接口描述

接口名称	方向	位宽	接口说明
FCLK_RESET0_N	I	1	系统复位
IRQ_F2P	IO	2	PS 中断信号， 与 PL 端 DMA 连接
S_AXI_HP0	IO	/	AXI_HP 从接口， 与 PL 端 DMA 连接
M_AXI_GP0	IO	/	AXI_GP 主接口， 与 DMA 的 S_AXI_LITE 连接
FCLK_CLK	O	/	PS 端系统时钟
M_AXI_GP0_ACLK	O	/	AXI_GP 接口的时钟控制信号接口
S_AXI_HP0_ACLK	O	/	AXI_HP 接口的时钟控制信号接口

下面对各个接口连接情况进行介绍：

IRQ_F2P 为中断控制信号接口，通过 Cancat IP 核连接到 AXI DMA IP 核中的中断信号接口上，当有数据需要接收或发送时，向 PS 端发出中断信号。

S_AXI_HP0 接口是高速数据交互通道从接口，通过 AXI Interconnect 模块的 M00_AXI 接口连接到 AXI DMA 模块的 AXI_S2MM 接口和 AXI_MM2S 接口上，分别实现 PL 内的流式消息切换内存映射消息传入 PS 端的内存中，PS 端内存中的内存映射消息切换为流式消息传入 PL 中。其中 AXI Interconnect IP 核可以进行不同数量的 AXI 主从接口的转换。

M_AXI_GP0 接口主要负责传递由 PS 端发起的配置信息和控制信息，通过 AXI Interconnect 的 S00_AXI 接口连接到 AXI_DMA 的 AXI_LITE 接口和 AXI BRAM Controller 的 S_AXI 接口上。

FCLK_CLK 为系统时钟接口，为整个 Zynq 平台的 PL 端提供工作时钟，且时钟频率可自定义切换，本工程使用的是 100MHz 的时钟频率。

4.3.2.2 AXI DMA 的配置与连接

如图 4-6 是 AXI DMA IP 核，其中各个接口的具体说明见表 4-3。



图 4-6 AXI DMA IP 核

表 4-3 为 AXI DMA 模块上经常使用到的重要接口介绍。图 4-6 中的 M_AXI_MM2S 口和 M_AXIS_MM2S 口都是该模块用来读取数据的接口，其连接情况见图 4-4，M_AXI_MM2S 经过 AXI Interconnect 与 Zynq-7000 PS 端的 S_AXI_HP0 接口连接，M_AXIS_MM2S 口与 AXI4-Stream Data FIFO_0 的 M_AXIS 连接。M_AXI_S2MM 和 S_AXIS_S2MM 是 DMA 写通道的接口，分别连接到 AXI Interconnect 模块的 S01_AXI 接口和 AXI4-Stream Data FIFO_0 的 S_AXIS 接口。S_AXI_LITE 是 DMA 重要的配置接口，用于传输 DMAC 的控制信息，本工程将其接到 AXI Interconnect 模块的 M01_AXI 接口。

表 4-3 AXI DMA IP 的主要接口说明

接口名称	方向	位宽	接口说明
M_AXI_S2MM	O	32	AXI4 类型的主接口，AXI DMA 模块会将接收到的流式消息格式切换为内存映射消息格式，然后经由此传出；
M_AXI_MM2S	O	32	AXI4 类型的主接口，AXI DMA 模块会将内存映射消息格式切换为流式消息格式，然后经由此传出；
M_AXIS_MM2S	O	32	AXI_Stream 类型的主接口，AXI DMA 模块会将内存映射消息格式切换为流式消息格式，然后经由此传出；
S_AXI_LITE	I	32	AXI4 类型的从接口，一般用于对 DMA 进行配置
S_AXIS_S2MM	I	32	AXI_Stream 类型的从接口，AXI DMA 模块会把流式消息格式切换为内存映射消息格式；
axi_resetn	I	1	复位信号

接口名称	方向	位宽	接口说明
m_axi_mm2s_aclk	I	1	时钟信号
m_axi_s2mm_aclk	I	1	
s_axi_lite_aclk	I	1	
mm_2s_introut	O	1	mm2s 中断输出
s2mm_introut	O	1	s2mm 中断输出

AXI DMA 模块中的一些参数名称和对这些参数的配置见图 4-7。本设计使能 Read Channel 并配置 Read Channel 的数量为 1，同时使能 Write Channel 并配置 Write Channel 的数量为 1。配置每个通道数据宽度为 32 位，同时配置缓存长度和地址位宽寄存器为 32 位。

图 4-7 AXI DMA 参数设置界面

图 4-8 为 AXI DMA 模块的地址配置，需要将 DMA 缓冲大小配置为 128MB，该 DMA 缓冲需要在 Linux 下驱动程序中进行合理的划分，用于存放帧数据和帧描述符。

Cell	Slave Interface	Base Name	Offset Address	Range	High Address
axi_dma_0					
Data_MM2S (32 address bits : 4G)					
processing_system7_0	S_AXI_HP0	HP0_DDR_LOWOCM	0x0000_0000	128M	0x07FF_FFFF
Data_S2MM (32 address bits : 4G)					
processing_system7_0	S_AXI_HP0	HP0_DDR_LOWOCM	0x0000_0000	128M	0x07FF_FFFF

图 4-8 AXI DMA IP 核地址分配

4.3.2.3 AXI BRAM Controller 和 Block Memory Generator 配置与连接

对于 PS 与 PL 之间传输少量、地址不连续且长度不规则的猝发数据的要求，PL 端提供了一种 BRAM 方式，适合进行软件对硬件寄存器的读写。要实现 PS 端

与 PL 端的 BRAM 传输，需要使用到 AXI BRAM 控制器模块和 Block Memory 生成器模块这两个 IP 核。

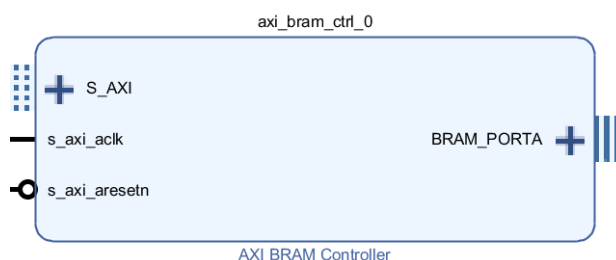


图 4-9 AXI BRAM Controller IP 核

AXI BRAM 控制器模块见图 4-9，在本工程中，此模块的 S_AXI 口接入到 ZYNQ-7000 PS 模块中的 M_AXI_GP0 口，进行 AXI 数据格式和 BRAM 数据格式的转换后，经过 BRAM_PORTA 口与 Block Memory 生成器模块的 BRAM_PORTA 口相连。Block Memory 生成器模块见图 4-10，节点卡功能部分通过其中的 BRAM_PORTB 接口访问 BRAM 中寄存器的值。

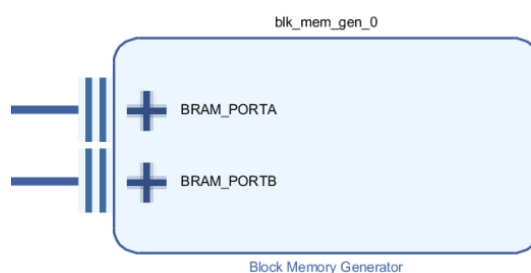


图 4-10 Block Memory Generator IP 核

接下来需要在地址配置界面配置 BRAM 的地址大小，如图 4-11 所示。由于节点卡的寄存器所占内存大小总共不超过 1MB，所以将 BRAM 的大小设置为 1MB，然后 Vivado 会自动分配该模块的 Offset Address 和 High Address，无需手动修改。

Cell	Slave Interface	Base Name	Offset Address	Range	High Address
> axi_dma_0					
▼ processing_system7_0					
▼ Data (32 address bits : 0x40000000 [1G])					
axi_bram_ctrl_0	S_AXI	Mem0	0x4000_0000	1M ▼	0x400F_FFFF
axi_dma_0	S_AXI_LITE	Reg	0x4040_0000	64K ▼	0x4040_FFFF

图 4-11 BRAM 自动分配的地址信息

4.4 节点卡驱动程序的设计

设备驱动程序即驱使硬件运行的程序，该部分代码的功能与其控制的具体硬件的功能密切相关。根据硬件平台的特点配置相关寄存器，接收并响应中断信号，实现软硬件数据交互。如果硬件平台上运行有操作系统，还需要完成地址映射等功能，同时驱动程序的具体设计需要根据该操作系统下特定的架构进行。

Linux 内核的结构见图 4-12，内核中的设备管理模块会对连接到系统中的外部设备进行分类处理，分为字符型设备、块数据设备和网络型设备^[42]。其中字符型设备和块数据设备在系统中被看作文件，通常以处理普通文件的方式处理设备，不同之处在于对字符型设备的读写只能顺序进行，而块数据设备则可随机读写。网络型设备用于实现硬件平台的远程通信功能，可以将硬件平台连接到不同的网络中，该设备的驱动程序承载了网络协议的实现。与其他两种类型不同的是，网络设备并不能体现出 Linux 中“一切都是文件”的思想^[45]。

按照其实现的具体功能划分，驱动程序可分为四个模块：初始化模块、数据发送模块、数据接收模块和控制模块。这几个功能模块与应用层和硬件得交互如图 4-13 所示，初始化模块主要完成设备和驱动的注册与绑定、分配内存资源、硬件寄存器初始化配置和注册中断等。应用程序可以将要发出的 FC 帧传输给数据发送模块，由该模块进行数据的进一步封装后交由硬件发出，并从数据接收模块接收节点卡硬件收到的 FC 帧做解析或提交给总控机。控制模块实现了应用程序和硬件的中间传输功能，可以响应应用层的各种控制信号，然后通过配置寄存器的方式通知硬件设备执行或者给出回应信号。

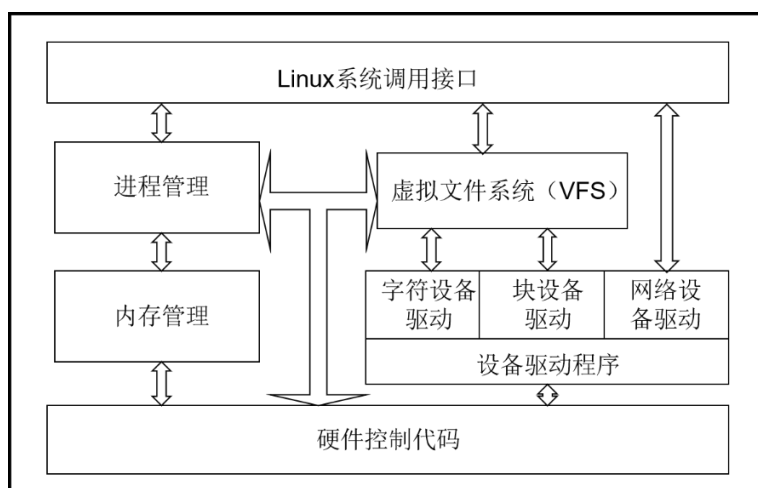


图 4-12 Linux 系统中软硬件跟驱动的关系

4.4.1 初始化模块

驱动初始化模块的设计相对比较固定，其具体流程是驱动的注册、建立字符设备模型、硬件寄存器的初始配置、DMA 缓存的申请、物理地址和虚拟地址之间的转换等。

其初始化过程中程序执行的流程如图 4-14 所示。

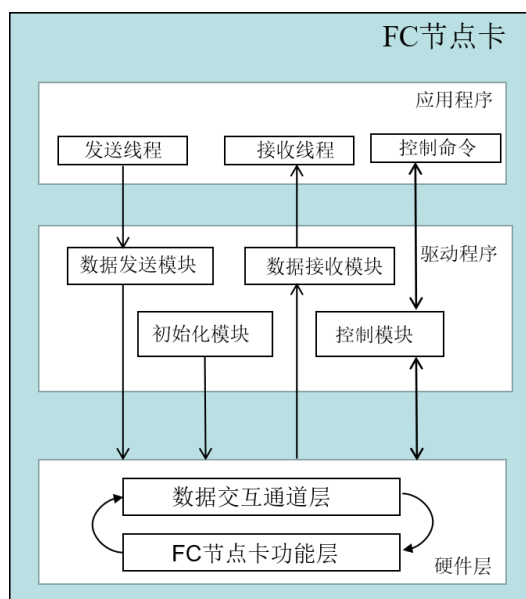


图 4-13 驱动跟软硬件的联系

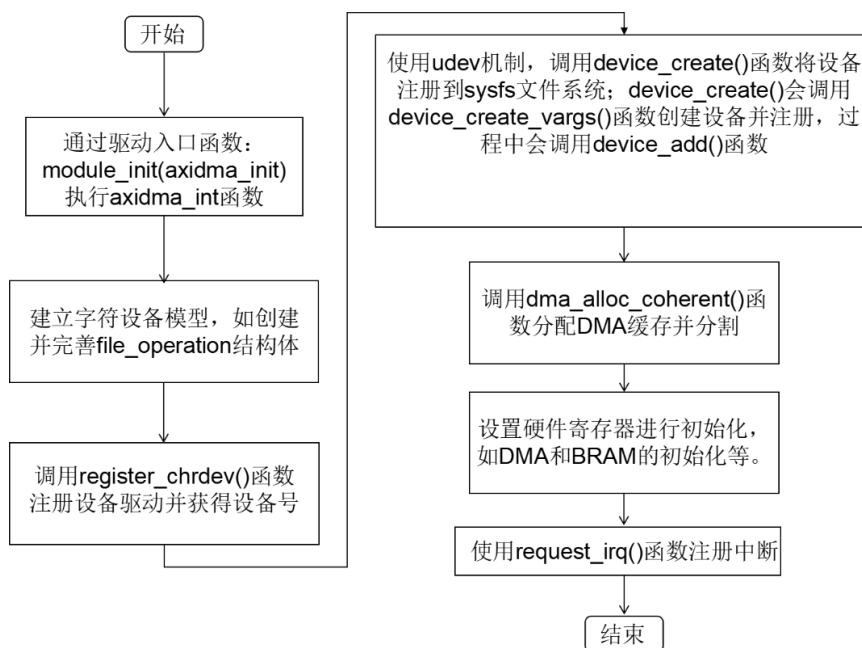


图 4-14 初始化流程图

Linux 源码在 linux/fs.h 头文件中定义的 register_chrdev()函数,函数原型如下:

```
int register_chrdev(unsigned int major,const char *name,const struct file_operations
*fops);
```

调用时需要传递三个参数,其中 major 是主设备号,在使用时该参数传入 0,函数会返回系统自动分配的设备号; name 参数应该传入该设备的名字^[46]。file_operations 是文件操作结构体,该结构体的创建和完善是整个字符设备驱动设计的主要任务。

file_operations 结构体中的成员函数是字符设备驱动和内核虚拟文件系统的接口,是应用层对 Linux 进行系统调用最终的执行者^[47]。所以应该实现 file_operations 中对应的应用层可能用到的系统调用,如 open()、close()、ioctl()和 mmap()等。file_operations 定义如下:

```
static struct file_operations axidma_fops=
{
    .owner=THIS_MODULE,
    //表示该结构的拥有者,初始化为 THIS_MODULE
    .open=axidma_open,
    //当应用层调用 open()时, axidma_open 函数会被调用
    .close=axidma_close,
    //当应用层调用 close()时, axidma_close 函数会被调用
    .unlocked_ioctl=axidma_ioctl,
    //当应用层调用 ioctl()时, axidma_ioctl 函数会被调用
    .mmap=axidma_mmap,
    //当应用层调用 mmap()时, axidma_mmap 函数会被调用
}
```

字符设备驱动注册函数 register_chrdev 调用的代码如下:

```
major=register_chrdev(0,"axidma_dev",&axidma_fops);
```

为了在模块加载时自动在/dev 目录下创建响应的设备节点,并在卸载模块时删除该节点,使用 device_create 函数创建该设备并注册到 sysfs 文件系统,device_create 函数原型如下:

```
struct device *device_create(struct class *class, struct device *parent,
                             dev_t devt, void *drvdata, const char *fmt, ...)
```

其中 struct class 结构体是 Linux 内核定义的,该结构体的变量对应一个类,内核同时提供了 class_create()函数来创建一个类,该类存放于 sysfs 下,代码如下:

```
axidma_class = class_create(THIS_MODULE,"axidma_dev");
axidma_device=device_create(axidma_class,NULL,MKDEV(major,0),NULL,"axi
dma_dev");
```

节点卡中帧的发送和接收都需要固定大小的 DMA 缓存，DMA 缓存的大小也决定了单次处理帧的数量，使用 `dma_alloc_coherent()` 函数进行 DMA 缓存的分配，然后为了满足发送和接收 FC-AE-ASM 帧的需要，划分出四个内存区域，包括发送描述符区域、接收描述符区域、ASM 帧发送缓冲区域、ASM 帧接收缓冲区域等。

初始化模块中另一个重要内容是相关硬件寄存器的初始化和虚拟地址映射，其中对 AXI DMA 中寄存器的配置决定了 AXI DMA 是否可正常工作，由 4.3.2.3 小节可知 AXI DMA 的基地址，需要使用 `ioremap()` 函数转换为虚拟地址，然后加上偏移地址定位特定的寄存器进行配置^[48]。表 4-4 中的寄存器为驱使 AXI DMA 模块正确工作的关键寄存器，从内存映射数据到流式数据方向上讲，具体配置过程如下：

1. 首先需要置 MM2S DMA 控制寄存器的 RS 位为 1，启动 MM2S 通道^[49]。
2. 设置 MM2S_DMCCR 寄存器的 IOC_IrqEn 位和 Err_IrqEn 位为 1 启动中断。
3. 将读取数据源地址写入到源地址寄存器 MM2S_SA。
4. 将传输字节数写入 MM2S_LENGTH 寄存器，该寄存器中的值会被 MM2S_AXI4 接口读取并传入 MM2S AXI4-Stream 接口，要求写入的值必须为非零值，否则无效。

表 4-4 为 AXI DMA 的主要寄存器的具体说明

寄存器名称	偏移地址（十六进制）	寄存器说明	
		比特位（名称）	比特位说明
MM2S_DMCCR （MM2S DMA 控制寄存器）	00h	0（RS）	控制 DMA 通道运行和停止； 0：停止 1：运行
		12 （IOC_IrqEn）	值为 1 时，允许 IOC_Irq 为设置了 IOC 位得描述符生成中断输出
		14 （Err_IrqEN）	值为 1 时使能错误中断

寄存器名称	偏移地址（十六进制）	寄存器说明	
		比特位（名称）	比特位说明
MM2S_DMASR (MM2S_DMA 状态寄存器)	04h	0 (Halted)	DMA 通道暂停位； 值为 1 时暂停 DMA 通道
		1 (Idle)	DMA 通道空闲位； 值为 1 时表示 DMA 通道空闲
S2MM_DMACR	30h	流式数据（S）转换（2）内存映射数据（MM）方向上的 DMA 控制寄存器	
S2MM_DMASR	34h	S2MM 方向上的 DMA 状态寄存器	
寄存器名称	偏移地址（十六进制）	寄存器说明	
		比特位（名称）	比特位说明
MM2S_SA	18h	MM2S 方向的 DMA 源地址寄存器，为该传输过程提供读取数据的源地址	
S2MM_DA	48h	S2MM 方向的 DMA 目标地址寄存器，为该传输过程提供写入数据的目标地址	
MM2S_LENGTH	28h	MM2S 方向的缓冲长度寄存器，提供该传输过程中需读出的字节长度	
S2MM_LENGTH	58h	S2MM 方向的缓冲长度寄存器，提供该传输过程中需写入的字节长度	

DMA 寄存器的配置代码如下：

```
#define AXIDMA_MM2S_ADD      0X40400000 //DMA 的 MM2S 通道基
地址
/*DMA MM2S 控制寄存器偏移地址*/
unsigned int  * mm2s_contr_offset;
#define MM2S_DMACR      0X00000000
```

```
/*DMA MM2S 状态寄存器偏移地址*/
```

```
unsigned int * mm2s_stat_offset;
```

```
#define MM2S_DMASR    0X00000004
```

```
/*DMA MM2S 源地址低 32 位地址*/
```

```
unsigned int * mm2s_low_add;
```

```
#define MM2S_SA        0X00000018
```

```
/*DMA 的 MM2S 通道传输字节长度*/
```

```
unsigned int * mm2s_tran_len;
```

```
#define MM2S_LENGTH    0X00000028
```

```
mm2s_cr = ioremap(AXIDMA_MM2S_ADD+MM2S_DMACR, 4);
```

```
mm2s_sr = ioremap(AXIDMA_MM2S_ADD+MM2S_DMASR, 4);
```

```
mm2s_sa = ioremap(AXIDMA_MM2S_ADD+MM2S_SA, 4);
```

```
mm2s_len = ioremap(AXIDMA_MM2S_ADD+MM2S_LENGTH,4);
```

由于 S2MM 通道对应寄存器的配置与 MM2S 通道的配置类似，这里不再累述。

4.4.2 数据发送模块

为了对 DMA 缓冲区中的 FC 帧进行高效的划分和读写，在驱动程序中使用发送和接收描述符来描述 ASM 帧的信息。本模块定义和使用的是帧发送描述符，它是一个固定大小的结构体，包括帧长度、帧地址和 ASM 帧头部分的各字段。对其定义如下：

```
struct send_desc_handle
{
    unsigned short len;    //帧长度
    unsigned int addr;    // 帧的地址
    unsigned short status;
    unsigned char  r_ctl;
    struct Port ID  d_id;
    unsigned char  cs_ctl;
    struct Port ID  s_id;
```

```

unsigned int    fc_params3;
unsigned char   seq_id;
unsigned char   df_ctl;
unsigned short  seq_cnt;
unsigned short  ox_id;
unsigned short  rx_id;
unsigned int    fc_params4;
unsigned int    msg_id;
unsigned int    security;
unsigned int    reserved2;
unsigned int    asm_params;
unsigned int    addr1;
unsigned int    idletime;
__int64        reserved3;
};

```

同时定义帧发送描述符队列控制结构体，其成员为读写指针和帧发送描述符 struct send_desc_handle 数组，定义如下：

```

struct send_desc_ctl
{
    int read_head;    //读指针
    int write_head;   //写指针
    struct send_desc_handle sdh[SEND_NUM];    //SEND_NUM 定义为 4096
};

```

读写指针以下标的方式记录描述符在 sdh 结构体数组的位置，其中读指针指向硬件当前读取到的帧描述符位置，写指针表示驱动应用程序要发送的帧描述符位置。读写指针初始化为 0。

整个发送的过程如下：

在该字符设备驱动的 file_operation 结构体中，axidma_fops->mmap 实现的是将帧发送缓冲区映射到进程的虚拟地址空间，使得应用程序直接访问该空间而无需在内核和应用间进行内存复制。axidma_fops->mmap 与 Zynq 上应用程序中的 void *mmap(void *addr,size_t length,int prot,int flags,int fd,off_t offset)函数对应^[50]。首先 Zynq-7000 收到需要发送的帧并填写帧对应的描述符，同时将帧描述符赋值到帧发送描述符队列的写指针指向的位置中，随后调用 ioctl 函数发送命令给驱

动程序的控制模块。节点卡驱动程序则进一步完善 FC 帧发送描述符中的字段，并完成地址大小端转换等。然后将帧发送队列控制结构体的写指针 `write_head` 加一表示数据已经写好，同时判断 `write_head` 的值是否达到 4096，如果达到则置 0。此时驱动程序的工作已经结束，硬件按照固定的时钟周期比较读写指针是否相等，如果写指针 `write_head` 大于读指针 `read_head` 这表示有新的 ASM 帧需要发送。硬件读取描述符队列中读指针 `read_head` 指向的描述符中的地址和长度，定位到需要发送的整个帧进行发送，然后将读指针 `read_head` 加一。具体 FC 帧发送方向程序流程见图 4-15。

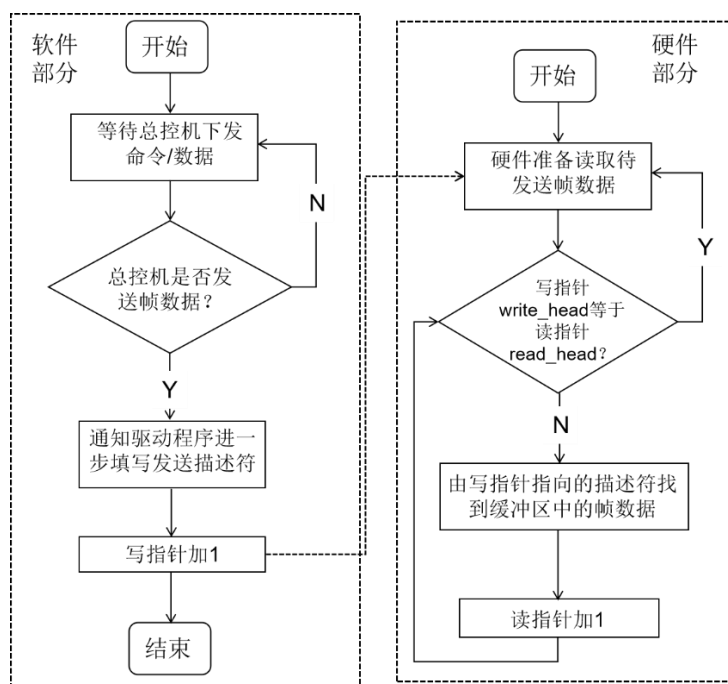


图 4-15 帧数据发送程序流程图

4.4.3 数据接收模块

数据接收模块的帧接收描述符 `struct receive_desc_handle` 包括帧长度、帧地址和 ASM 帧头字段，FC 帧接收描述符队列控制结构体的具体定义如下。

```

struct receive_desc_ctl
{
    int read_head;    //读指针
    int write_head;   //写指针
    struct receive_desc_handle rdh[RECEIVE_NUM]; //RECEIVE_NUM 定义为
4096
};

```

同样的，帧接收描述符队列控制结构体 `struct receive_desc_ctl` 中包括了读写指针和帧接收描述符 `struct receive_desc_handle` 的数组。读写指针以下标的方式记录 `rdh` 结构体数组的位置，其中读指针指向硬件当前读取到的帧描述符位置，写指针表示驱动应用程序要发送的帧描述符位置。读写指针初始化为 0。

整个接收的过程如下：

当 Zynq 中的 PL 端通过 FC 接口收到帧后，通过 DMA 将帧数据顺序存放在 DMA 接收缓冲区，同时对接收描述符队列控制结构体变量中的 `rdh[write_head]` 结构体进行赋值，`status` 写入帧接收的端口号和类型等，`len` 写入帧长，`addr` 写入帧的物理地址，然后将写指针加一并触发硬件中断信号。此时硬件的工作就完成了，CPU 会进入中断处理程序进行下一步的处理，首先比较帧接收描述符队列控制结构体中的读写指针是否相等，如果写指针大于读指针，则说明硬件有新的 FC 帧发来，驱动程序对对应描述符做进一步处理，如大小端转换和物理地址转换等，然后按照 `MessageID` 的值将描述符复制到对应 `MessageID` 的缓冲区中。最后是通知应用程序处理帧的过程，驱动和应用程序之间的交互，同样使用描述符队列的方式进行，驱动程序将配置好的描述符队列复制到与应用程序交互的一块内存空间中，然后将应用接收描述符队列中的写指针加一来通知应用程序新帧数据到来。应用程序用同样的方式处理此帧，包括根据对应描述符取到整个帧数据，返回发送给总控机，并将应用描述符队列中的读指针加一等。其具体的 FC 帧接收方向上的程序流程见图 4-16。

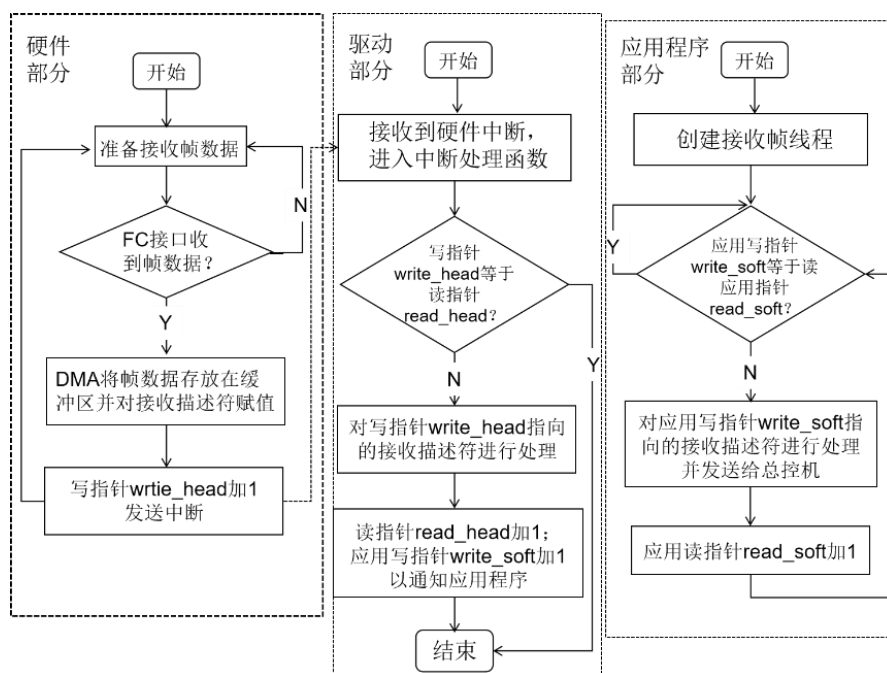


图 4-16 帧数据接收程序流程图

4.4.4 控制模块

FC 节点卡驱动程序中的控制模块实现了一种从应用层到硬件的控制流。通过该模块，上层应用程序可以直接控制驱动程序，也可以通过驱动程序间接控制硬件。

驱动程序对硬件的控制有两种可用的方式：一是通过内存映射，直接存储访问技术可以使得硬件读写内存中的数据，而 Linux 下的驱动程序不能直接访问物理地址空间，需要提前进行内存映射后才能读写该内存。二是通过控制硬件寄存器的方式，同 4.4.1 小节中对 AXI DMA 寄存器配置类似，在驱动程序中可以通过访问对应寄存器的地址空间对相应硬件的控制。也可以调用 `mmap` 函数进行地址映射实现对硬件寄存器的控制，前提是需实现文件操作结构体中的相应函数功能。

本工程中驱动程序的控制模块是使用 Linux 的 `ioctl` 机制实现。应用程序调用 `ioctl` 函数和驱动程序进行交互，驱动程序根据其参数不同做出进一步处理，首先需从 `ioctl()` 的函数原型理解 `ioctl` 机制^[51]。`ioctl()` 的函数原型为：

```
int ioctl(int fd,unsigned long cmd,...);
```

其中参数 `fd` 为该设备的文件描述符，通过调用 `open()` 函数获得，参数 `cmd` 为控制码，通常定义为 `IOCTL_XXXXX` 的形式，区分应用程序对驱动发出的不同命令，每个命令需要在驱动程序中实现。后面的扩展参数是需要传递给驱动程序的数据或者驱动程序返回的数据，为可选参数。当调用成功且正确执行命令后，函数返回 0 值，否则返回值为 -1，如果想进一步了解错误所在，可以查看变量 `ERRORNO` 的值。整个控制模块的 `ioctl` 机制交互结构图如图 4-17 所示：

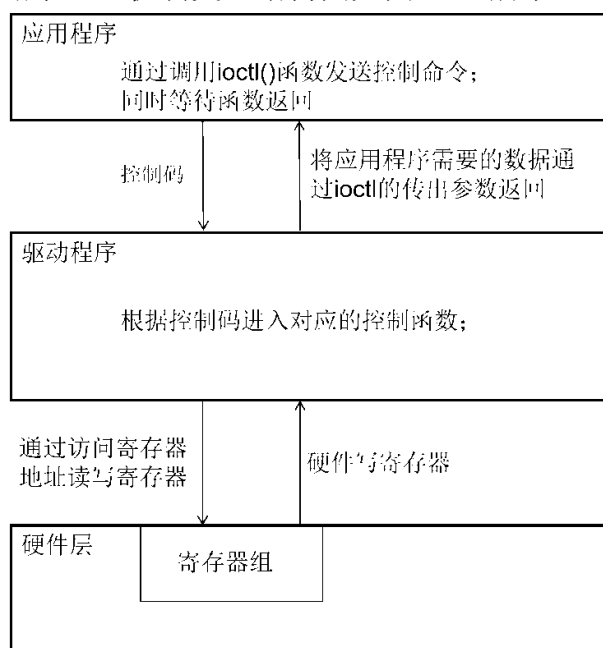


图 4-17 控制模块交互结构

为实现应用程序对硬件设备的控制，本模块定义了一些必要的控制码，具体说明如表 4-5 所示。

表 4-5 部分控制码列表

控制码	功能
IOCTL_RESET	FC 节点卡复位
IOCTL_BUILD_LINK_PORT0	0 端口建立/断开链路
IOCTL_BUILD_LINK_PORT1	1 端口建立/断开链路
IOCTL_EXCHANGE_SPEED	切换链路速率
IOCTL_LINK_SPEED_PORT0	获取 0 端口链路速率
IOCTL_LINK_SPEED_PORT1	获取 1 端口链路速率
IOCTL_ASM_COUNT_PORT0	统计 0 端口 ASM 收发消息数
IOCTL_ASM_COUNT_PORT1	统计 1 端口 ASM 收发消息数
IOCTL_MMAP_ADDR	虚拟地址映射
IOCTL_REQ_ASM_ADDR	获取发送 ASM 地址
IOCTL_SEND_ASM	发送 ASM 消息

4.5 节点卡应用程序的设计

节点卡应用程序的主要功能是与总控机建立通信，同时不断接收总控机的命令并解析。然后根据不同命令通知驱动程序配置寄存器来操作硬件，或者将总控机需要的信息或数据帧返回。以此实现总控机对 FC 节点卡的控制。

总控机和节点卡在不同的硬件环境上运行，需要用到适用于远程通信的套接字(socket)编程。为了适应使用 socket 编程的各种系统的不同要求，有三种 socket 类型可供选择：本地套接字(SOCK-RAW)、流式套接字(SOCK-STREAM)和数据报套接字(SOCK-DRAM)。由于总控机和节点卡之间要发送大规模的数据并且对于可靠性要求很高，所以选择基于 TCP 协议的流式套接字进行网络通信。本工程使用的 TCP 通信为了区分不同命令，以进行高效处理，在应用层自定义了新的通信格式，具体格式定义见图 4-18。

TCP报头	TCP数据段		
	Cmd(4Byte)	Len(4Byte)	Payload

图 4-18 自定义 TCP 数据段格式

其中 Cmd 是控制码; Len 表示 Payload 部分的数据长度, 单位为 Byte; Payload 表示有效传输数据字段。Cmd 高位的 2 个 Byte 表示总控机发出命令的具体模块, 低位 2 个 Byte 的不同数值表示不同的控制命令。

当把 FC 节点卡启动并接入网络后, 可以使用命令查询该节点卡的 IP 地址, 然后在总控机的控制界面输入该节点卡的 IP 地址。总控机得到节点卡的 IP 地址后, 立即与节点卡建立 TCP 连接, 然后按照以上通信格式封装要传输的数据, 发送给节点卡。其与节点卡的通信流程见图 4-19。

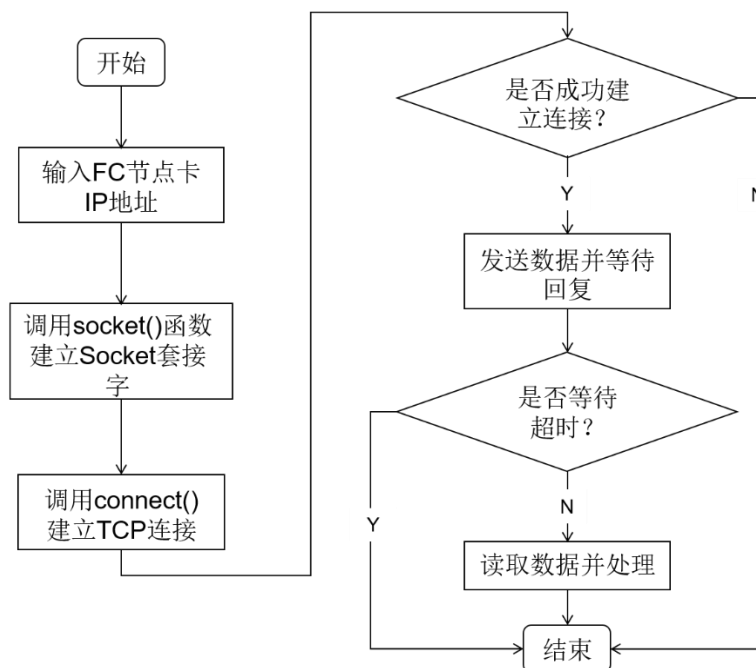


图 4-19 总控机 TCP 通信过程

节点卡应用程序启动后首先找到 axidma 设备, 打开该设备并通过驱动程序进行初始化, 设置速率和冗余模式等参数。然后会和总控机建立 TCP 连接, 等待总控机发送来的数据并解析。收到数据后按照约定的格式解析出响应的 Cmd 命令字、数据长度和有效数据, 然后做对应的处理。其具体程序的执行流程见图 4-20。

4.6 本章小结

本章首先对航电地面仿真系统的架构进行了简要的介绍, 同时给出了基于 Zynq 的 FC 节点卡整体设计架构。FC 节点卡的软件设计与实现是本章的重点内容, 分为三个部分: 数据交互通道部分、节点卡驱动程序设计和节点卡应用设计部分。对于一个完整节点卡系统来说, 还有节点卡 FPGA 功能部分, 由于不是本文的重点内容, 仅做了简要介绍。

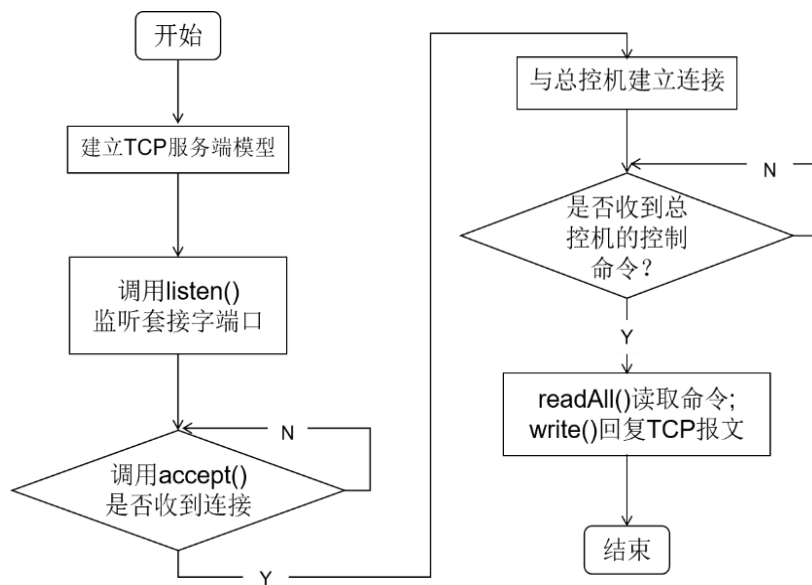


图 4-20 节点卡 TCP 通信过程

第五章 测试与分析

在完成了基于 Zynq 平台的 Linux 实时化改造和 FC 节点卡设计之后,本章进行测试与分析。测试的内容主要有两大方面:移植到 Zynq 平台的嵌入式实时 Linux 的实时性能和基于此实时平台的 FC 节点卡测试。

5.1 Linux 实时性测试

5.1.1 Cyclicttest 测试

有一些常用的 Linux 操作系统实时性能测试工具,比如 Intlat、Lmbench 和 Cyclicttest 等。这些工具都可以测试 Linux 内核实时化改造的有效性,其中 Cyclicttest 最常用于实时补丁的测试。同时, Cyclicttest 是 rt-tests 下的一个高精度测试工具,可以对高精度时钟进行测试,所以选择使用 Cyclicttest 进行实时性测试。

Cyclicttest 通过测试 Linux 内核的任务响应延迟来测试系统的实时性。即重复测量进程或线程的实际响应时间,和预计响应时间对比得到差值,最后得出 Linux 系统的实时性能延迟的统计数据。Cyclicttest 是多线程运行的程序,其程序的运行流程也相对简单。首先运行一个普通进程,然后启动一定数量的实时线程,实时线程的数量根据需要可以通过 Linux 命令行传入,由于这些线程为实时线程,调度策略为 Linux 内核中默认的实时调度策略 SCHED_FIFO。实时线程通过时钟中断被周期唤醒,然后记录下被唤醒的时间。随后,求出实际的唤醒时间减去预先设定的唤醒时间的值并记录,不断循环记录延迟值直到达到用户设置的循环次数后,对记录的延时进行统计,最后显示在 Bash 中循环测试任务响应延迟的最小值、最大值和平均值。由于实时线程被中断唤醒,其实际获得执行之前,首先要经历 2.2.2 小节中的中断响应时间,在中断处理中调度新的实时线程时又会产生上下文切换时间。故 Cyclicttest 测试的任务响应延迟中包括了上下文切换时间和中断响应延迟时间,由于与 Linux 实时性相关的指标为最大任务响应延迟,所以最需要关注和比较的是其最大值。

对于 Cyclicttest 的使用,需要了解其参数的含义。在串口命令行键入 cyclicttest -help 可以看到 Cyclicttest 支持的所有参数命令和详细说明。下面对常用的参数进行介绍:参数“-p”加数字表示设置实时线程的优先级,最大为 99,一般设置为 90 以上;参数“-n”表示选择定时精度更高的 nanpsleep 定时器;参数“-m”表示参

数“-i”加数字表示设置线程间的基本间隔，单位为 μs ，不设置则默认为 $1000\mu\text{s}$ ；参数“-l”加数字表示设置循环次数，不设置则默认为无限循环。

5.1.2 测试结果分析

在有一定 CPU 负载的情况下分别对标准 Linux 和 Preempt-RT Linux 进行循环测试，实时线程优先级均设置为 95，表 5-1 和图 5-1 为测试结果统计和对比柱状图。

表 5-1 Cyclicttest 测试结果

操作系统	最小延迟 (单位: μs)	平均延迟 (单位: μs)	最大延迟 (单位: μs)
标准 Linux	12	20	967
Preempt-RT Linux	13	22	69

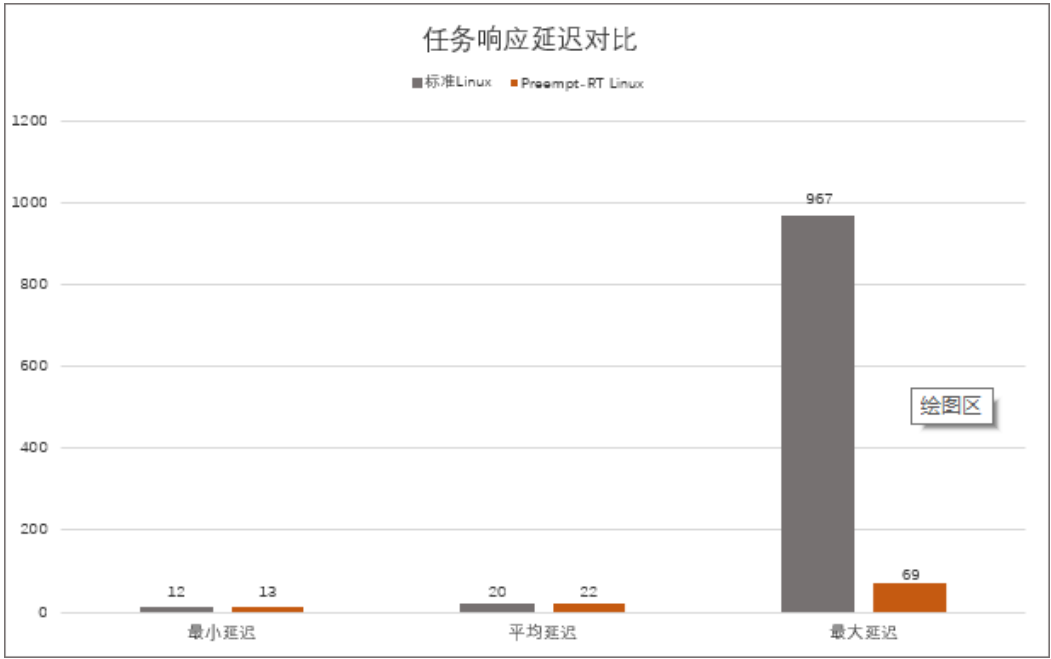


图 5-1 Cyclicttest 测试结果柱状图

从图 5-1 的测试结果中可明显看出，Preempt-RT Linux 的平均响应延迟较标准 Linux 增加了 $2\mu\text{s}$ ，但值得注意的是，Preempt-RT Linux 在最大响应延迟上表现极为出色，只有 $69\mu\text{s}$ ，在 $100\mu\text{s}$ 范围之内，达到了预期要求。而标准 Linux 在有 CPU 负载的情况下，很难保证最大响应延迟在一个可接受的范围。这也印证了 Preempt-RT Linux 的实时性有了明显提升。

5.2 节点卡软件测试

本节将对基于 Zynq 实时平台设计的 FC-AE-ASM 节点卡进行测试，测试主要分为两个方面：功能测试和性能测试。以验证基于 Zynq 平台的 FC-AE-ASM 节点卡在硬件、驱动和应用程序上能否协同工作，在保证传输性能达到要求的情况下，完成基本的 FC 节点卡功能。

5.2.1 测试平台介绍

节点卡测试平台的结构如图 5-2 所示。测试平台中的硬件设备包括 Zynq-7000 开发板、宿主机、总控机和监控卡。如 2.3.1 小节的表 2-2 所示，Zynq 平台开发板 MZ7035 上有一路千兆以太网接口，使得 MZ7035 可以和总控机接入同一个局域网内进行通信。同时 MZ7035 还配置了 4 个 SFP+接口，可以插上光电转换模块接入 FC 网络中进行光纤通信。本工程使用该开发板上其中 2 个 SFP+接口作为 FC 接口完成 FC-AE-ASM 帧的发送与接收，由于 FC-AE-ASM 协议中的网络节点都是对等的，且单张节点卡有两个 FC 端口，所以可以进行回环测试。

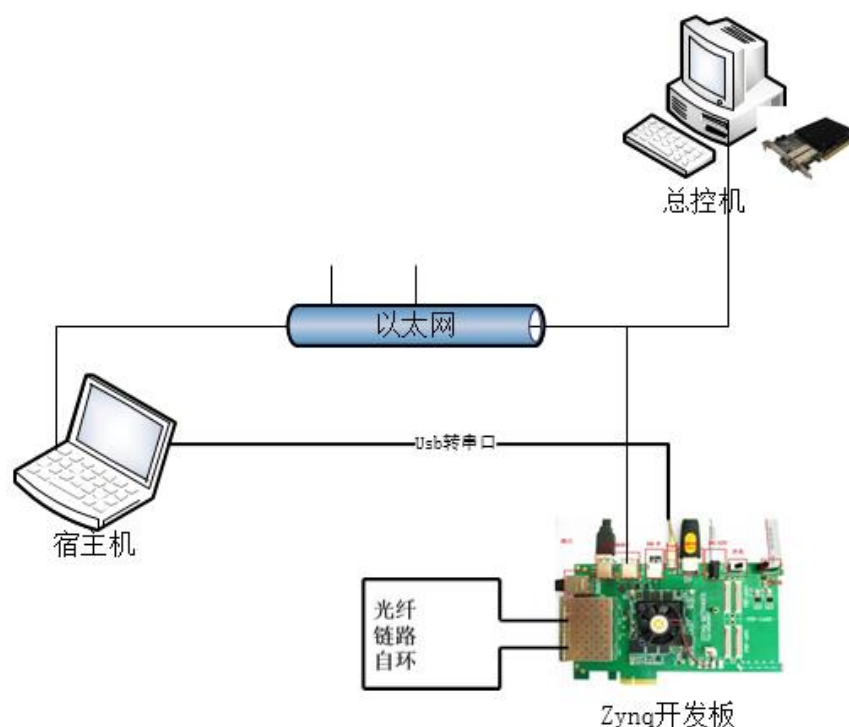


图 5-2 测试平台结构

其中使用的 Zynq 开发板 MZ7035FD 如图 5-3 所示。

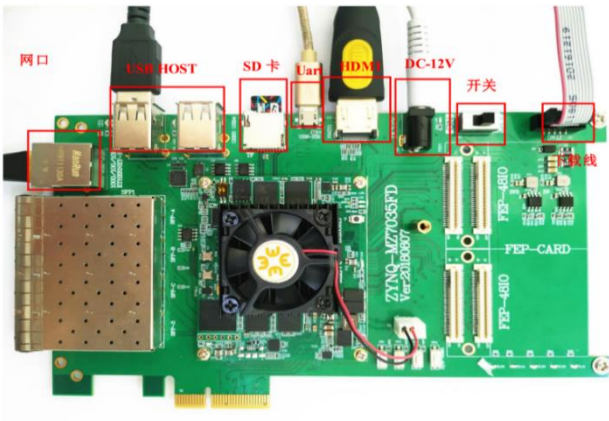


图 5-3 Zynq 开发板 MZ7035

5.2.2 测试方案

由 2.5.2 小节可知，FC-AE-ASM（Anonymous Subscriber Message）协议是一种匿名传输协议，消息接收方不需要知道确切的消息发送方，只需要确定每一条 FC-AE-ASM 消息都是来自于单一序列的单项交换。由于本文设计的基于 Zynq-7000 Soc 的 FC 节点卡是双冗余传输的，即提供两个 FC 接口连接功能，可以模拟 FC-AE-ASM 网络中的两个节点进行传输，故本次测试只需要使用一块 Zynq 平台开发板。对于 FC 节点卡软件部分的测试主要包括帧收发功能测试和性能测试。在总控机上打开本节点卡的管理界面，如图 5-4 所示。



图 5-4 ASM 节点卡管理界面

对 FC-AE-ASM 帧收发进行测试，首先需要生成发送帧头和帧数据部分，如图 5-5 所示。

能，然后点击“快速生成调度表”会生成两个端口的调度表，调度表会计算每个ASM 帧发送的时间点。最后点击发送按钮进行 ASM 帧的发送。



图 5-6 ASM 帧配置

发送完成后可以在统计界面查看两个端口（Port0 和 Port1）分别发送和接收数帧的个数，如图 5-7 所示。

	CRC错误帧	SOF错误帧	EOF错误帧	发送帧数	接收帧数
Port0个数统计	0	0	0	18000	18000
Port1个数统计	0	0	0	18000	18000

图 5-7 ASM 帧收发统计

如图 5-8 所示，在接收端解析帧界面可以看到接收到的 ASM 帧的具体内容，对比发现接收到的数据与发送的数据完全一致，数据传输正确。

5.2.4 传输性能测试

传输性能测试中主要测试三种链路模式下的发送速率，即一秒内发出的 FC 帧数据量，单位为 MB/s，实际测得的平均发送速率达到各种链路下理论最大速率的 50%即可满足工程的预期需求，即带宽占用率大于等于 50%。节点卡功能模块统计每秒的数据量然后更新到传输速率寄存器中，应用层可以将值读出并发送给总控机显示。

解析		清除		统计帧数: 4000							
	帧类型	D_ID	S_ID	OX_ID	RX_ID	SEQ_ID	SEQ_CNT	F_CTL	TYPE	DF_CTL	帧长/满帧
1	ASM	030000	000000	0980	FFFF	00	0008	3A0018	49	00	2100/2400
2	ASM	030000	000000	0980	FFFF	00	0004	3A0018	49	00	2100/2400
3	ASM	030000	000000	0980	FFFF	00	0009	3A0018	49	00	2100/2400
4	ASM	030000	000000	0980	FFFF	00	0005	3A0018	49	00	2100/2400
5	ASM	030000	000000	0980	FFFF	00	000A	3A0018	49	00	2100/2400
6	ASM	030000	000000	0980	FFFF	00	000B	3A0018	49	00	1524/2400
7	ASM	030000	000000	0981	FFFF	00	0006	3A0018	49	00	2100/2400
8	ASM	030000	000000	0981	FFFF	00	0007	3A0018	49	00	2100/2400
9	ASM	030000	000000	0981	FFFF	00	0008	3A0018	49	00	2100/2400
10	ASM	030000	000000	0981	FFFF	00	0009	3A0018	49	00	2100/2400
11	ASM	030000	000000	0981	FFFF	00	000A	3A0018	49	00	2100/2400

Fibre Channel(FC)		BC B5 36 36	04 03 00 00	00 00 00 00	49 3A 00 18	00 00 00 08	09 80 FF FF	00 00 40 00	11 11 00 00	00 00 00 00	00 00 00 00
SOF = BC853636		AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21
Rctl = 04		AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21
D_id = 030000		AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21
Cs_ctl = 00		AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21
S_id = 000000		AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21
Type = 49		AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21
Fctl = 3A0018		AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21
Seq_id = 00		AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21
Df_ctl = 00		AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21
Seq_cnt = 0008		AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21
Ox_id = 0980		AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21
Rx_id = FFFF		AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21	AB CD 43 21

图 5-8 接收端帧解析

由于 FC-AE-ASM 帧的最大长度为 2148 字节，且帧与帧之间的间隙为 24 字节，同时考虑硬件的 8B/10B 编码，以 4G 的链路速率（具体为 4.25Gbit/s）为例，计算理论最大发送速率为：

$$4.25 \times \frac{2148}{2148+24} \times \frac{8}{10} = 3.3624 \text{ Git/s} = 420.30 \text{ MB/s} \quad \text{公式 (5-1)}$$

同样的方法可以计算得到 1G 和 2G 链路速率下的理论最大发送速率为 105.08MB/s 和 210.15MB/s。具体的测试方案为：在每种链路速率下均进行 20 次发送测试，并求其平均值，发送配置 FC-AE-ASM 帧数据长度均为 24000 字节，周期均为 2000，其他配置项和帧收发功能测试中的相同。对发送速率进行统计，见表 5-2。

表 5-2 FC-AE-ASM 传输速率测试统计

链路速率 (Gbit/s)	理论最大发送速率 (MB/s)	20 次发送速率平均值 (MB/s)	带宽占用率 (%)
1	105.08	75.73	72.07
2	210.15	148.68	70.75
4	420.30	275.11	65.46

由于 Zynq 平台的嵌入式特性,在硬件资源上与独立高性能 FPGA 和基于 x86 架构的 PC 设备相比,存在一定的性能差距,主要表现在 CPU 运算能力和内存读写速度等方面,故在帧传输速率上会有一定的局限性。本节点卡在 1Gbit/s、2Gbit/s 和 4Gbit/s 三种链路速率下的发送速率平均值分别为 75.73MB/s、148.68MB/s 和 275.11MB/s,带宽占用率分别为 72.07%、70.75%和 65.46%,均超过了理论链路最大速率的 50%,达到了 FC-AE-ASM 帧传输的预期指标和工程要求。

5.3 本章小结

本章首先使用 Cyclicttest 实时测试工具对 Zynq 平台上经过 Preempt-RT 改造的 Linux 内核和标准 Linux 内核进行实时性能测试,比较得出实时化改造后的 Linux 内核的实时性确有提高。然后对基于 Zynq 平台设计的 FC 节点卡软件进行测试,主要测试的内容包括帧收发功能测试和性能测试。测试结果表明,基于 Zynq 平台的 FC 节点卡软件达到了预期的工程需求。

第六章 总结与展望

6.1 总结

将 Zynq-7000 全可编程平台应用到航电地面仿真系统具有重大意义，也是未来的一大发展趋势。系统中的 FC 节点卡作为 FC 网络中重要的通信设备，目前普遍是基于“x86+FPGA”的分立式架构实现的。本文提出了一种基于 Zynq 平台的 FC 节点卡设计方案，由于 Zynq 平台的软硬件架构集成在同一个芯片内，且使用高效的片内总线连接，极大的提高了硬件利用率和软硬件通信的稳定性。同时该平台具有 Xilinx 开发工具套件的良好支持，使得软硬件设计更加便捷与灵活，极大缩短了开发周期。同时为了避免资源受限的嵌入式系统对节点卡性能上的影响，需要在 Zynq 平台上搭建实时操作系统，本文采用了实时化改造标准 Linux 系统的方案实现这一目标。主要内容可归纳如下：

1. 详细介绍了全文涉及到的一些背景知识和关键技术，为论文中具体设计的阐述提供了理论支持。主要内容包括实时系统介绍、实时操作系统概述、软硬件协同开发平台 Xilinx Zynq 的介绍、AXI 总线和 DMA 技术的介绍。最后分析了用在航电地面仿真系统中的 FC 协议和子协议 FC-AE-ASM。

2. 对标准 Linux 内核的实时性进行研究，重点分析了限制 Linux 实时性的因素。然后提出业内常用的两种实时化方案：双内核法和直接修改内核源码法，最后研究 Preempt-RT 对 Linux 内核优化的关键技术，对 Preempt-RT 进行配置并进行移植测试。

3. 实现基于 Zynq 实时平台的 FC 节点卡软件部分的设计。首先对包含 FC 节点卡的整个航电地面仿真系统进行介绍，明确 FC 节点卡在整个系统中发挥的作用。然后提出基于 Zynq 平台的 FC 节点卡的整体设计架构，最后详细描述了本文的三大设计重点：数据交互通道部分设计、节点卡驱动程序设计和节点卡应用程序设计。

4. 对基于 Zynq-7000 SoC 的实时 Linux 系统进行实时性能测试，并与标准 Linux 内核进行对比，结果表明其最大任务响应延迟有了显著减小，验证了实时化改造的有效性。然后对基于 Zynq 平台的 FC 节点卡软件部分进行功能和性能测试，证明了本设计在性能满足工程要求的同时，实现了 FC-AE-ASM 节点卡的基本收发功能。

6.2 展望

由于实际测试条件和时间的限制,本文设计的基于 Zynq 平台的 FC 节点卡还处在初步阶段,还可以在以下几个方面进行改进:

1. 目前,该 FC 节点卡仅支持 FC-AE-ASM 协议,不支持同样广泛使用的 FC-AE-1553 协议,可以考虑实现对于 FC-AE-1553 协议的兼容,使单个节点卡设备可以适应更多的实际应用场景。

2. 虽然在嵌入式领域 Linux 系统的应用较多,并且在进行实时化改造后,其实时性也会显著提高。但是在一些极为关键的领域中实时性还是有所欠缺,对于涉及重大人身安全、涉及重要机密和重大财产的系统,使用专业性更强的一些商用操作系统可能更为合适的。同时 Zynq 平台也支持一些其他操作系统,在今后的研究中可以考虑对其他硬实时操作系统进行移植和调试,然后和实时 Linux 系统进行对比择优。

致 谢

行文至此，三年的硕士研究生生活也步入尾声。在电子科技大学求学的时间里，学习了很多，经历了很多，也成长了很多。

首先感谢我的导师谢军老师，回想起第一次独自赶到成都时，谢老师和善且耐心的介绍教研室的研究方向和项目，给了我很大的鼓励和信心。后来从研究方向的确定、课题的进行到论文的撰写，谢老师都给予了耐心的指导。同时感谢涂晓东老师、孟中楼老师和朱晓东老师，三位老师在这三年里对我的学习和项目都给予了莫大的帮助。

感谢教研室的师兄师姐在项目 and 论文撰写上的热心帮助，你们不厌其烦的指导和答疑，使我少走了很多弯路。感谢教研室的五位同学，你们优秀的专业素养一直深深激励着我进步。

感谢我的母亲，求学三年，您是我最大的牵挂和最坚强的后盾。感谢王曦彤同学的陪伴和鼓励。

最后，感谢百忙之中评审本论文的老师，谢谢！

参考文献

- [1] 郑赞,王勇,杨若海.软硬件协同设计综述[J].中国集成电路,2002(10):45-49
- [2] 张治国.ARM 嵌入式系统综述[J].科技资讯,2007(21):107-108
- [3] C. Maxfield. Xilinx Zynq-7000 All Programmable SoCs for trusted systems[J]. Xilinx, Zynq-7000
- [4] 唐洁.基于实时操作系统的支持时间触发通信的 FC-AE 节点卡软件设计[D].电子科技大学,2018
- [5] 樊帆. FC-AE-ASM 协议桥模块研制[D].哈尔滨工业大学,2015
- [6] 尹叶丹.基于嵌入式 Linux 的 CAN-TCP/IP 网关的研究与实现[D].武汉理工大学,2004
- [7] 沈涛.Linux 内核实时性以及网络实时性的研究[D].南京邮电大学,2013
- [8] 刘剑,仲宇,王琦.嵌入式 Linux 实时性改造技术综述[J].航天控制,2018,36(02):93-97
- [9] 于国龙.嵌入式操作系统的实时性分析与研究[D].兰州理工大学,2010
- [10] Kopetz, Hermann. Real-Time Systems: Design Principles for Distributed Embedded Applications[M]. Springer Publishing Company, Incorporated, 2011.
- [11] 汤根生.基于 CPS 的实时系统的面向方面的测试方法[D].广东工业大学,2011
- [12] S. Baskiyar, N. Meghanathan. A Survey of Contemporary Real-time Operating Systems[J]. Informatica (Slovenia), 2005, 2(2):233-240
- [13] 刘宝宁.基于多核平台的实时混合任务调度算法研究[D].武汉理工大学,2013
- [14] J. A. Stankovic, R. Rajkumar. Real-Time Operating Systems: Special Anniversary Issue (Guest Editors: John A. Stankovic, Wolfgang Halang, Kim-Fung Man)[J]. Real-Time Systems, 2004, 28(1):237-253
- [15] 王伟.基于 ARM 网络语音 AGC 系统的研究及实现[D].中南大学,2009
- [16] B. B. Brandenburg. Scheduling and locking in multiprocessor real-time operating systems[J]. University of North Carolina at Chapel Hill, 2011
- [17] R. P. Kar. Implementing the Rhealstone real-time benchmark[J]. Dr Dobbs Journal, 1990, 15(4):46-55
- [18] A. C. Heursch, A. Horstkotte, H. Rzehak. Preemption concepts, Rhealstone Benchmark and scheduler analysis of Linux 2. 4 Concepts to improve the pre-emption of the kernel. 2001.
- [19] 王元凯.实时操作系统基准评测系统中的高精度时间测量技术研究[D].浙江大学, 2012
- [20] 卢娥,贾桂芬,程越巍,等.基于嵌入式实时操作系统的弹载计算机软件设计[J].计算机测量与控制, 2014, 22(7)

- [21] 万欣宇,赵明.上下文切换延迟测量的一种改进方法[J].信息与电脑(理论版).2013,01:109
- [22] HCC Embedded; HCC Embedded's Extensive Suite of Advanced Middleware Now Available for Xilinx Zynq-7000 SoCs[J]. Information Technology Newsweekly, 2017
- [23] 倪峰.基于 Zynq7000 嵌入式平台的 Flash 验证系统设计与实现[D].苏州大学,2014
- [24] A. Techcon. Xilinx Zynq-7000 receives product of the year ACE award[J]. Datasheets Com, 2012.
- [25] 何宾.Xilinx All Programmable Zynq-7000 SoC 设计指南[M].清华大学出版社,2013.5
- [26] 王一帆,邱昆,李永刚.FC-AE-ASM 协议综述[C].2009 中国西部青年通信学术会议.2009
- [27] 崔晓南.基于 ARM 的分布式工业信号采集模块的研究[D].河北工业大学,2013
- [28] 李小群,赵慧斌,叶以民,孙玉芳.一种基于时钟粒度细化的 Linux 实时化方案[J].计算机研究与发展,2003(05):734-740
- [29] 王琼.嵌入式 Linux 系统的实时性改造[J].电脑知识与技术.2010,14(22):6229-6230,6249
- [30] RobertLove,陈莉君,康华.Linux 内核设计与实现 (原书第 3 版) [M].北京:机械工业出版社,2011
- [31] 吴章金. Linux 实时抢占补丁的研究与实践[D].兰州大学,2010
- [32] 郑金爽.基于 RTAI 的嵌入式实时 Linux 的研究及实现[D].吉林大学,2010
- [33] 王涛,王立德,周洁琼,申萍.基于交换式以太网的列车通信网络实时性研究[J].铁道学报,2015,37(04):39-45
- [34] K. S. Roy,K. Gowthami. Implementation of Xenomai Framework in GNU/Linux Environment to run applications in a Real Time Environment[J]. Indian Journal of Science and Technology,2016,9(17)
- [35] 魏欣.基于 Xenomai 的 Td-SCDMA 接收机软件设计及并行优化[D].厦门大学, 2013
- [36] 韩东奇,陈永恒.Linux 实时性分析与改善方法测试[J].科技创新导报,2018,15(13):159-160
- [37] 王朴,周晴.基于龙芯 1E 的嵌入式 Linux 实时性的优化与可靠性设计[J].微电子学与计算机,2019,v.36;No.426(11):17-21
- [38] 刘俞,刘恒,章懿雯.嵌入式实时操作系统中优先级反转问题的研究[J].齐齐哈尔大学学报(自然科学版),2017,33(01):5-8
- [39] 徐潇. linux 嵌入式下的多媒体应用和开发[D].上海交通大学,2005
- [40] 殷文锋.基于 FPGA 的光纤扰动检测系统硬件设计与实现[D].西安电子科技大学,2019
- [41] AndrewN.Sloss, DominicSymes. ARM 嵌入式系统开发:软件设计与优化[M]. 北京航空航天大学出版社, 2005.8
- [42] 郑伟,王钦若,吴乃优.Linux 内核空间设备驱动程序的开发[J].微计算机信息,2003
- [43] 李木国,黄影,刘于之.基于 FPGA 的 PCIe 总线接口的 DMA 传输设计[J].计算机测量与控制, 2013,21(001):233-235,249

- [44] 陆佳华,江舟,马岷机械工业出版社.嵌入式系统软硬件协同设计实战指南:基于 Xilinx Zynq[电子资源][M].机械工业出版社, 2013
- [45] 宋宝华. LINUX 设备驱动开发详解: 基于最新的 LINUX4.0 内核[M].机械工业出版社, 2015,6
- [46] 王聚生.基于嵌入式系统的前端软件框架的设计与实现[D].兰州大学
- [47] 薛佳音.基于 CK-CPU 嵌入式平台的 Linux 系统移植与 USB 驱动开发[D].哈尔滨工业大学
- [48] 潘旭明.基于多接口 SSD 的软件设计[D].黑龙江大学
- [49] 陈聪.基于 Zynq 的雷达信号处理系统软件模块的设计与实现[D].西安电子科技大学
- [50] 夏振新.基于嵌入式 ARM 和 Linux 的电阻点焊监控器的研制[D].哈尔滨工业大学
- [51] N. Stakhanova, S. Basu, J. Wong. On the symbiosis of specification-based and anomaly-based detection[J]. Computers & Security, 2010, 29(2):253-268

攻读硕士期间的科研项目和成果

参与的科研项目

- [1] Linux 实时化研究
- [2] 基于 Zynq 平台的节点卡软件设计

获奖情况

- [1] 2018 年获得研究生三等奖学金
- [2] 2019 年获得研究生一等奖学金
- [3] 2020 年获得研究生二等奖学金